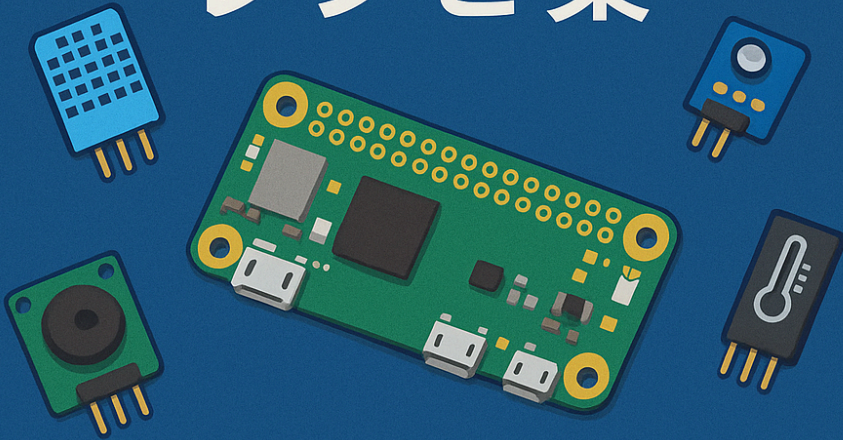


CHIRIMEN 対応デバイス レシピ集



CHIRIMEN Open Hardware コミュニティ

目次

CHIRIMEN 対応デバイスレシピ集	8
本書について	8
本書の特徴とおすすめポイント	8
前提条件	8
直ぐに開発を始めたい方	9
CHIRIMEN とは	10
CHIRIMEN のメリット	11
コミュニティについて	11
入門用	12
hello-real-world	12
Lチカ (GPIO OUTPUT)	12
ギヤードモーター part1	14
ギヤードモーター part2	16
ギヤードモーター part3	18
ギヤードモーター part4	20
ギヤードモーター part5	22
MOSFET P718D モジュール	24
MOSFET D4184 モジュール	26
GPIO 関連	28
基本的な GPIO 操作	28
GPIO スイッチ	28
GPIO スイッチ (GPIO INPUT)	31

GPIO タッチセンサー	31
タッチセンサー	34
GPIO スイッチ 2 (GPIO POLLING INPUT)	34
タッチセンサー	36
MPR121 静電容量センサ(12ch)	36
カメラ	38
GPIO スイッチによるカメラ撮影 (GPIO-Camera)	38

環境センサー	44
温度・湿度センサー	44
AHT10 温湿度センサー	47
AHT20 温湿度センサー	49
HTU21D 温湿度センサー	51
SHT30 温湿度センサー	53
SHT40 温湿度センサー	55
気圧センサー	57
BMP180 大気圧温度センサー	57
BMP280 温度・気圧センサー	59
BME280 温度・湿度・気圧センサー	61
空気品質センサー	63
ENS160 空気質センサ	63
SGP40 ガスセンサー	66
CO2 センサー	68
SCD40 CO2 センサー	68
光センサー	70
BH1750 光センサー	70
LTR390 UV センサー	72
TSL2561 Grove Digital Light Sensor 光センサー	74
TSL2591 照度センサー	76
VEML6070 UV センサー	78
色センサー	80
TCS34725 カラーセンサー	80
S11059 デジタルカラーセンサー	82

モーション・位置センサー	84
加速度・ジャイロセンサー	84
ADXL345 Grove Accelerometer 3 軸加速度センサー	84
ICM20948 9 軸センサー	86
MPU6050 3 軸ジャイロ 3 軸加速度 複合センサー	88
MPU9250 3 軸ジャイロ 3 軸加速度 3 軸 磁気複合センサー ..	90
距離センサー	92
GP2Y0E03 測距センサー 40 mm - 0.1 m.....	92
VL53L0X レーザー測距センサー 30 mm - 2 m.....	94
VL53L1X レーザー距離センサー	96
角度センサー	98
AS5600 磁気式角度センサ.....	98
ジェスチャーセンサー	101
PAJ7620 Grove Gesture ジェスチャー認識センサー	101
雷センサー	103
AS3935 雷センサー	103
ディスプレイ・LED	107
OLED ディスプレイ	107
SSD1306 OLED ディスプレイ	107
SSD1308 Grove OLED ディスプレイ	109
LED ドライバー	111
HT16K33 LED マトリクスドライバ.....	111
NeoPixel	117
Neopixel LED	117
電子ペーパー	122
WAVESHARE20471 複合センサーボード	122

モーター制御	125
ステッピングモータードライバー	125
A4988 ドライバによるステッピングモータ制御	125
H ブリッジ	128
HBridge1	128
PCA9685 16 チャンネル PWM ドライバ.....	131
PWM コントローラー	134
PCA9685 16 チャンネルサーボモーター PWM ドライバー..	134
アナログ・デジタル変換.....	136
ADC	136
ADS1015 12 ビット AD コンバータ	136
ADS1115 16bit ADC 差動入力によるロードセルの使用.....	138
ADS1115 16bit ADC.....	140
PCF8591 8bit AD,DA コンバーター	142
電流センサー.....	144
INA219 電流センサー	144
オーディオ	146
オーディオプレーヤー.....	146
DFPlayer Mini	146
音声録音	149
ISD1820 ボイスレコーダ (GPIO OUTPUT)	149
その他.....	151
GPS.....	151
Serial GPS	151
サーマルカメラ	154
AMG8833 サーモグラフィー	154
赤外線温度センサー	156
MLX90614 赤外線温度センサー	156

リモート制御	158
カメラ	158
GPIO スイッチによるカメラ撮影 (remote_Camera)	158
GPIO	161
リモートLチカ	161
リモートオンオフセンサ	164
H ブリッジ	167
リモートモータ	167
PWM	171
リモートサーボモータ	171
センサー	174
リモート温湿度センサ	174
リモート温湿度センサ2	177
リモート距離センサ	180

CHIRIMEN 対応デバイスレシピ集

本書について

本書は、Raspberry Pi と各種センサー（温度・湿度・加速度など）を活用した電子工作やプログラミングに関心のある方を対象とした技術同人誌です。CHIRIMEN コミュニティで動作確認済みのサンプルコードや回路図を豊富に掲載しており、初心者から経験者まで幅広くご利用いただけます。

- 公式サイト：CHIRIMEN Raspberry Pi Zero W チュートリアル^{*1}
- 書籍：ちりめんばいぜろちゅーとりある^{*2}

上記いずれかと併せて読んでいただくとより一層理解が深まります。

本書の特徴とおすすめポイント

- 各センサーごとに分かりやすい電子回路図を掲載しています。
- 動作確認済みで、コピー＆ペーストですぐに試せるサンプルコードを掲載しています。
- コミュニティで検証された信頼性の高いパーツを紹介しています。
- 電子工作や IoT に興味のある方、Raspberry Pi を使ってみたい方、センサーの基礎から実践まで学びたい方に最適です。

前提条件

本書の内容をお試しいただくには、以下の環境を推奨します。

デバイス

- Raspberry Pi Zero W（Wi-Fi 搭載モデル）

1. <https://tutorial.chirimen.org/pizero/>

2. <https://x.gd/OmMZJ>

主な購入先：

- KSY：Raspberry Pi Zero WH [RASPIZWHSC0065]^{*3}
- SWITCH SCIENCE：Raspberry Pi Zero WH [RPI-ZERO-WH]^{*4}

必要な Node.js ライブラリ

- I2C 制御用ライブラリ：node-web-i2c^{*5}
- GPIO 制御用ライブラリ：node-web-gpio^{*6}
- Node.js のバージョン^{*7}は、v22.x 以上を推奨します。

OS

- Raspbian OS をカスタマイズした CHIRIMEN Lite^{*8} を使用します。
- ダウンロードは公式サイト^{*9}をご参照ください。

直ぐに開発を始めたい方

- 公式サイト：CHIRIMEN Raspberry Pi Zero W チュートリアル^{*10} にアクセスして下さい。
- 書籍：ちりめんぱいぜろちゅーとりある^{*11}をご購入ください。

3. <https://x.gd/HytS7>

4. <https://x.gd/KF2aF>

5. <https://www.npmjs.com/package/node-web-i2c>

6. <https://www.npmjs.com/package/node-web-gpio>

7. <https://nodejs.org/ja/about/previous-releases>

8. <https://github.com/chirimen-oh/chirimen-lite>

9. <https://github.com/chirimen-oh/chirimen-lite/releases>

10. <https://tutorial.chirimen.org/pizero/>

11. <https://x.gd/OmMZJ>

CHIRIMEN とは

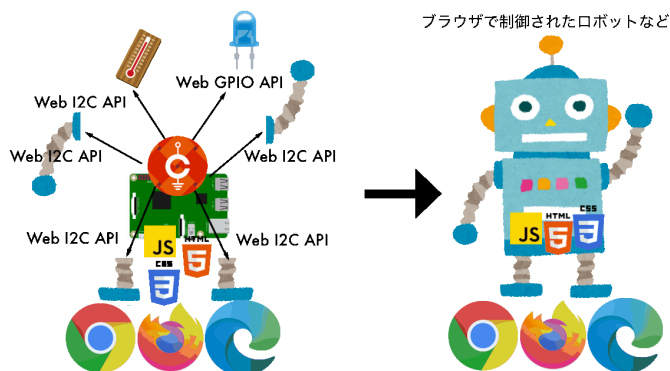


図 1: CHIRIMEN_pf

CHIRIMEN は Web ブラウザや Node.js の JavaScript からハードウェアを制御するプロトタイピング環境です。

CHIRIMEN コミュニティと W3C の「Browsers and Robotics コミュニティグループ」^{*1}では、JavaScript で Web アプリから電子パーツを直接制御できる低レベルハードウェア制御 API (WebGPIO API^{*2} や WebI2C API^{*3} など) の標準化に向けての検討・提案と、それらの API を Raspberry Pi などの開発ボード上で使うプロトタイプ環境 (CHIRIMEN 環境) を実装しています。

Web ページや Node.js 上の JavaScript から直接ハードウェアを制御できるため、既存の Web 技術や知識を活かしつつ、一つのプログラムで画面表示やサービス連携、電子パーツの制御を統合できます。開発環境もローカルエディタだけではなく Web ブラウザ上のオンラインエディタで動くようにしており、サンプルコード集^{*4}の通り配線シタブを開くだけですぐに多数の電子パーツ^{*5}を試していただくとよいでしょう。

1. <https://www.w3.org/community/browserobo/>

2. <http://browserobo.github.io/WebGPIO>

3. <http://browserobo.github.io/WebI2C>

電子パーツ制御のために専用のツールや開発環境を用意したり、複数の言語やプログラムを連携させたり、独特のフレームワークを学習・利用したりする必要はありません。Web 標準技術をベースとしており、素早いプロトタイピングやプログラミング初学者の IoT 学習に最適です。

CHIRIMEN のメリット

- 広く使われている標準技術を学習できるので
 - 学習のハードルが低い
 - 得たスキルが広く長く役立つ
 - インターネットでノウハウを検索しやすい
- Web 技術を活用するので
 - WWW のサービスと簡単に連携できる
 - ユーザーインターフェースやコンテンツを簡単に作れる
 - Web ブラウザを使って開発できる

コミュニティについて

何か不明点や困ったことがあれば CHIRIMEN コミュニティの Slack^{*6} や Github^{*7} の各リポジトリの issues でご連絡・ご相談ください。

- Slack^{*8}
- Github^{*9}
- X^{*10}
- Facebook^{*11}

4. <https://github.com/chirimen-oh/chirimen.org/tree/master/pizero/src/esm-examples>

5. <https://tutorial.chirimen.org/partslist>

6. <http://chirimen-oh.slack.com/>

7. <https://github.com/chirimen-oh/>

8. <http://chirimen-oh.slack.com/>

9. <https://github.com/chirimen-oh/>

10. https://x.com/chirimen_oh

11. <https://www.facebook.com/groups/chirimen/>

入門用

hello-real-world

Lチカ (GPIO OUTPUT)

配線図

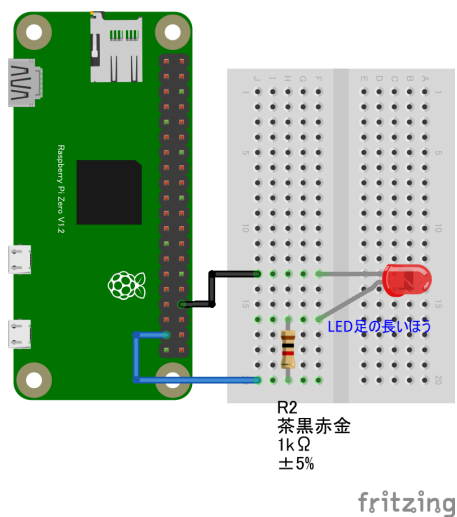


図 1: 配線図

GPIO PORT26 に LED+抵抗を繋ぎます。

CHIRIMEN ドライバのインストール

- 不要です。

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

async function blink() {
  const gpioAccess = await requestGPIOAccess();
  const port = gpioAccess.ports.get(26);

  await port.export("out");

  for (;;) {
    await port.write(1);
    await sleep(1000);
    await port.write(0);
    await sleep(1000);
  }
}

blink();
```

ギヤードモーター part1

配線図

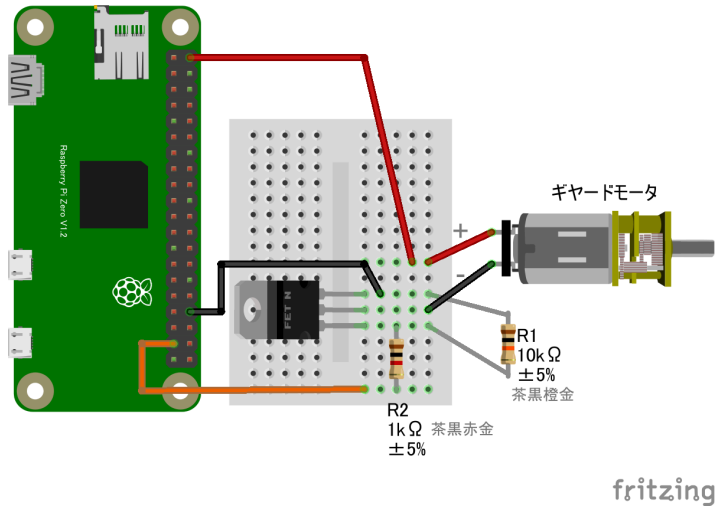


図 1: 配線図

GPIO PORT26 にモーター制御回路を繋ぎます。コードはLチカと共通です

CHIRIMEN ドライバのインストール

- 不要です。

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

async function blink() {
  const gpioAccess = await requestGPIOAccess();
```

```
const port = gpioAccess.ports.get(26);

await port.export("out");

for (;;) {
  await port.write(1);
  await sleep(1000);
  await port.write(0);
  await sleep(1000);
}
}

blink();
```

ギヤードモーター part2

ギヤードモーター (GPIO OUTPUT)外部電源を使用する場合

配線図

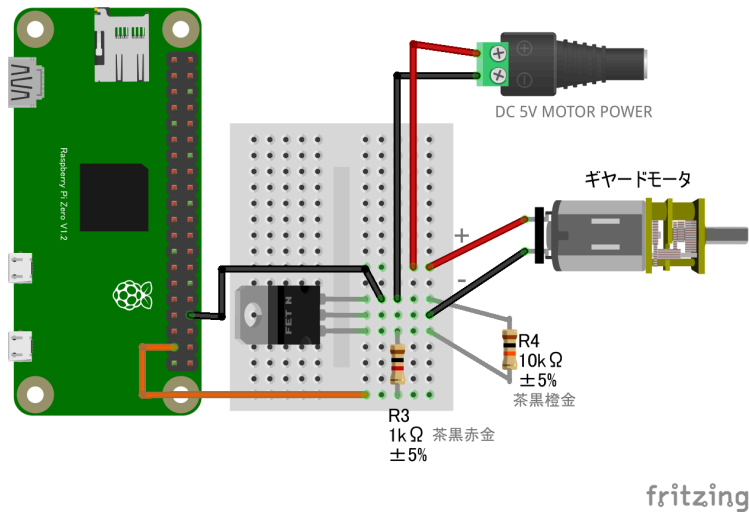


図 1: 配線図

5V 以外の DC 電源も使用可能です。

CHIRIMEN ドライバのインストール

- 不要です。

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
async function blink() {  
  const gpioAccess = await requestGPIOAccess();  
  const port = gpioAccess.ports.get(26);  
  
  await port.export("out");  
  
  for (;;) {  
    await port.write(1);  
    await sleep(1000);  
    await port.write(0);  
    await sleep(1000);  
  }  
}  
  
blink();
```

ギヤードモーター part3

配線図

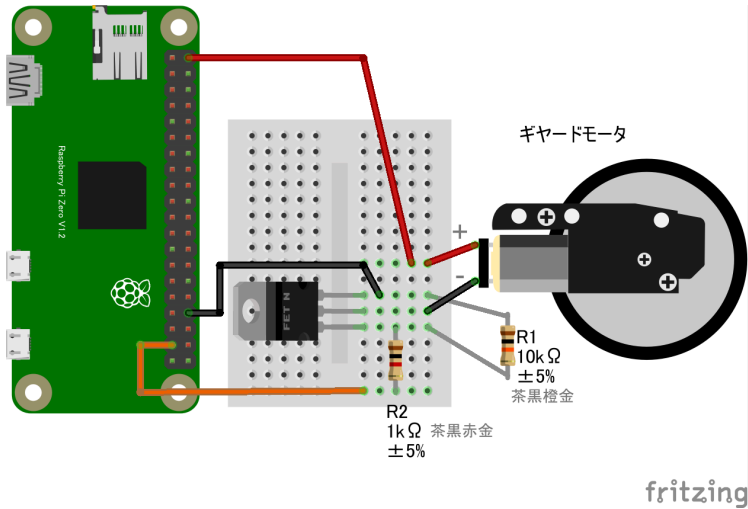


図 1: 配線図

CHIRIMEN ドライバのインストール

- 不要です。

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

async function blink() {
  const gpioAccess = await requestGPIOAccess();
  const port = gpioAccess.ports.get(26);
```

```
await port.export("out");

for (;;) {
  await port.write(1);
  await sleep(1000);
  await port.write(0);
  await sleep(1000);
}

blink();
```

ギヤードモーター part4

配線図

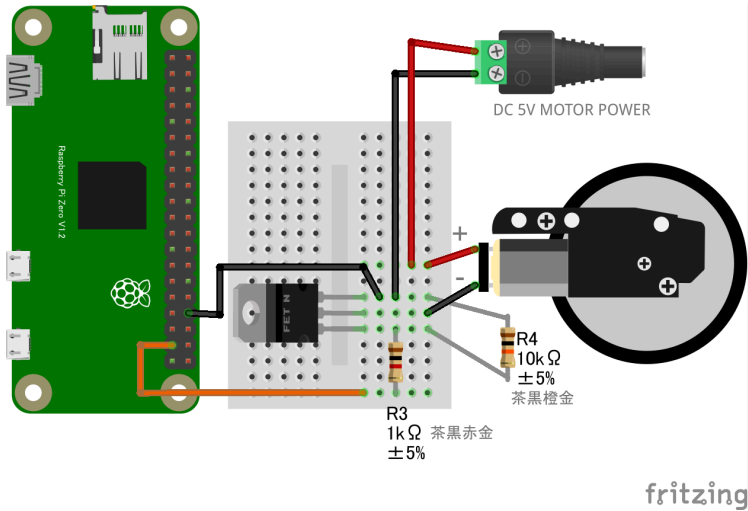


図 1: 配線図

CHIRIMEN ドライバのインストール

- 不要です。

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

async function blink() {
  const gpioAccess = await requestGPIOAccess();
  const port = gpioAccess.ports.get(26);
```

```
await port.export("out");

for (;;) {
  await port.write(1);
  await sleep(1000);
  await port.write(0);
  await sleep(1000);
}

blink();
```



```
for (;;) {  
    await port.write(1);  
    await sleep(1000);  
    await port.write(0);  
    await sleep(1000);  
}  
}  
  
blink();
```

MOSFET P718Dモジュール

MOSFET モジュール基板を使用する場合 P718D モジュール

配線図

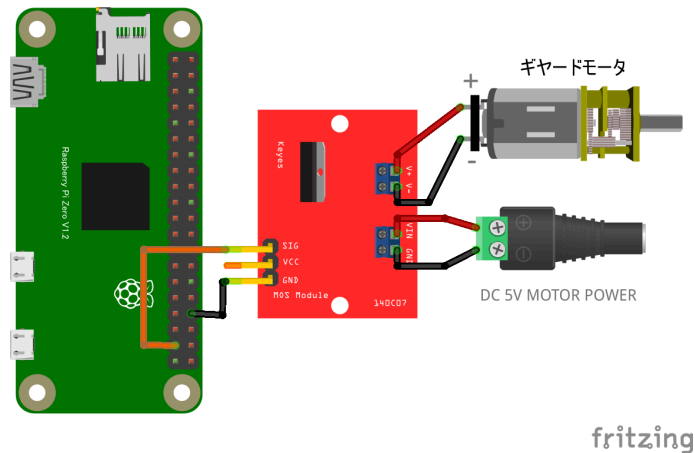


図 1: 配線図

CHIRIMEN ドライバのインストール

- 不要です。

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

async function blink() {
  const gpioAccess = await requestGPIOAccess();
  const port = gpioAccess.ports.get(26);
```

```
await port.export("out");

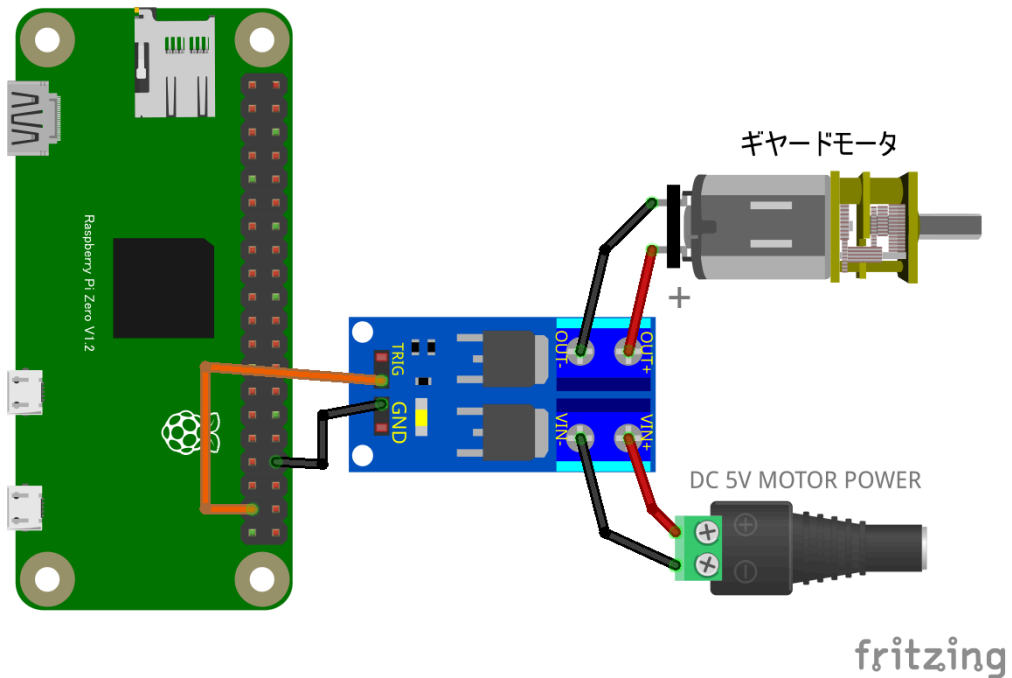
for (;;) {
  await port.write(1);
  await sleep(1000);
  await port.write(0);
  await sleep(1000);
}

blink();
```

MOSFET D4184モジュール

MOSFET モジュール基板を使用する場合 D4184 モジュール

配線図



5V 以外の DC 電源も使用可能です。

CHIRIMEN ドライバのインストール

- 不要です。

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
async function blink() {  
  const gpioAccess = await requestGPIOAccess();  
  const port = gpioAccess.ports.get(26);  
  
  await port.export("out");  
  
  for (;;) {  
    await port.write(1);  
    await sleep(1000);  
    await port.write(0);  
    await sleep(1000);  
  }  
}  
  
blink();
```

GPIO 関連

基本的な GPIO 操作

GPIO スイッチ

GPIO スイッチ and LED又はギヤードモーター (GPIO INPUT/OUTPUT)

配線図1 (スイッチ+LED)

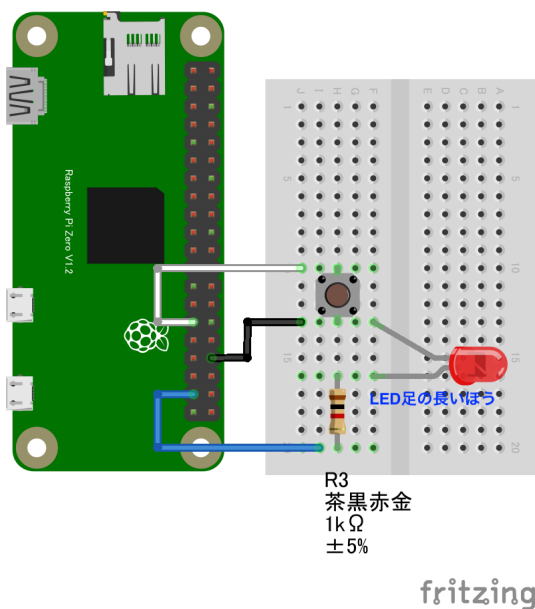


図1: 配線図

GPIO PORT5 にスイッチ、GPIOPORT26 に抵抗と LED を繋ぎます。

配線図 2 (スイッチ+ギヤードモーター)

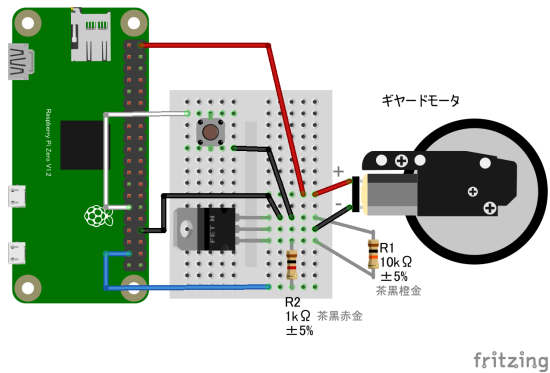


図 2: 配線図

GPIO PORT5 にスイッチ、GPIOPORT26 にモーター制御回路を繋ぎます。

CHIRIMEN ドライバのインストール

- 不要です。

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
let port2;

async function startGpio() {
  const gpioAccess = await requestGPIOAccess();
```

```
port2 = gpioAccess.ports.get(26);
await port2.export("out");

const port = gpioAccess.ports.get(5);
await port.export("in");
port.onchange = showPort;
}

function showPort(ev){
  console.log(ev.value);
  if (ev.value==0){
    port2.write(1);
  } else {
    port2.write(0);
  }
}

startGpio();
```

GPIOスイッチ (GPIO INPUT)

配線図

GPIO PORT5 に繋がります

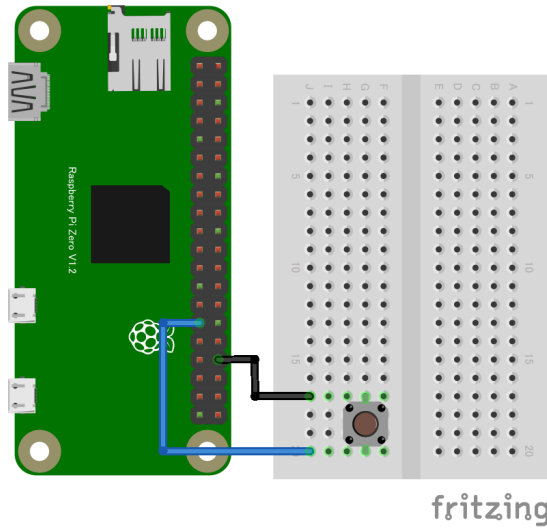


図 1: 配線図

GPIO タッチセンサー

TP223 を用いたタッチセンサー(タッチスイッチ)を使います。プログラムコードは通常のスイッチと同じです。

配線図

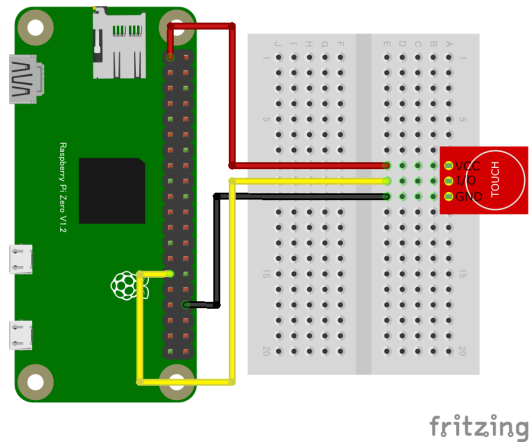


図2: 配線図

GPIO PORT5に繋がります。3.3V電源を用います。指でのタッチだけでなく金属の近接も感知します。(1~2mmに接近すると検知)。なお、接近させる金属がセンサーのピンに接触してショートしないようテープで金属を被覆し絶縁しておく安全です。

CHIRIMEN ドライバのインストール

- 不要です。

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

async function switchCheck() {
  const gpioAccess = await requestGPIOAccess();
  const port = gpioAccess.ports.get(5);

  await port.export("in");
```

```
port.onchange = showPort;

}

function showPort(ev){
    console.log(ev.value);
}

switchCheck();
```

タッチセンサー

GPIOスイッチ2 (GPIO POLLING INPUT)

配線図

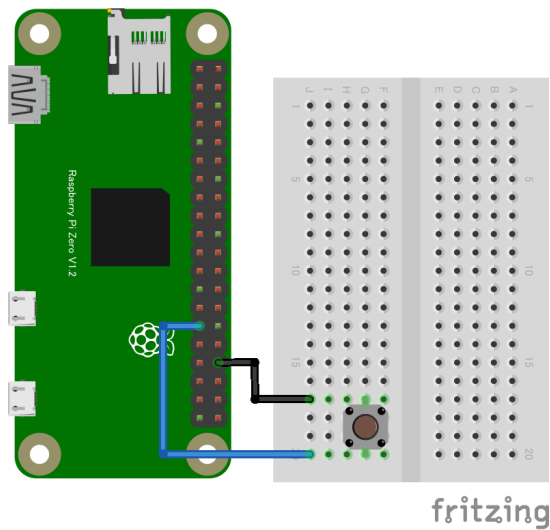


図 1: 配線図

CHIRIMEN ドライバのインストール

- 不要です。

GPIO PORT5 に繋がります

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve,
```

```
msec));  
  
async function switchCheck() {  
  const gpioAccess = await requestGPIOAccess();  
  const port = gpioAccess.ports.get(5);  
  
  await port.export("in");  
  
  for(;;){  
    const v = await port.read();  
    console.log(v);  
    await sleep(300);  
  }  
}  
  
switchCheck();
```

タッチセンサー

MPR121 静電容量センサ(12ch)

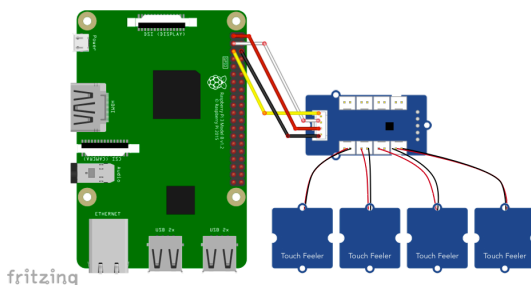


図 1: 回路図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/grove-touch
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import GroveTouch from "@chirimen/grove-touch";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  var i2cAccess = await requestI2CAccess();
  var port = i2cAccess.ports.get(1);
  var touchSensor = new GroveTouch(port, 0x5a);
  await touchSensor.init();
  for (;;) {
    var ch = await touchSensor.read();
    console.log(JSON.stringify(ch));
    await sleep(100);
  }
}
```

カメラ

GPIO スイッチによるカメラ撮影 (GPIO-Camera)

配線図

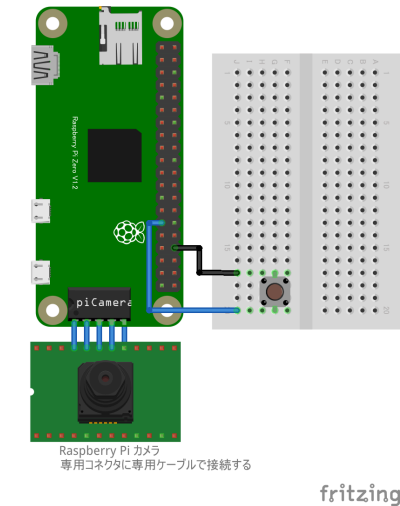


図 1: 配線図

タクトスイッチは GPIO PORT5 に繋がります。 カメラは専用コネクタに専用ケーブルを使って接続し、更にセットアップが必要です。

カメラのセットアップと動作確認

Raspberry Pi のカメラ^{*1}を API で直接操作する pi-camera-connect^{*2}を使った方法です。 Pi-Camera^{*3}を使った方法(gist はこちら^{*4})より、大幅に高速に画像が取得できることを確認しています。

1. <https://www.raspberrypi.com/documentation/accessories/camera.html>

2. <https://www.npmjs.com/package/pi-camera-connect>

3. <https://github.com/stetsmando/pi-camera>

準備

- Raspberry Pi カメラモジュール
 - 例 1:KEYESTUDIO カメラモジュール^{*5}、例 2^{*6}、例 3^{*7}
- Zero 用ケーブル～上のモジュールは添付されているようです。
 - 無い場合は 別途調達^{*8}
- 接続のしかた^{*9} : Zero は専用ケーブルでつなぎます

Note

利用可能なカメラモジュールは v1、v3 です。Camera Module v2 には未対応です。また Raspberry Pi Zero 用 CHIRIMEN v1.4.0 未満をお使いの場合、Camera Module v3 には未対応です。Raspberry Pi Zero 用 CHIRIMEN v1.4.0 以上^{*10} をお使いください。

カメラの動作テスト

以下のコマンドで画像ファイルが保存されます:

```
raspistill -v --width 640 --height 480 -o test.jpg
```

Note

Raspberry Pi Zero 用 CHIRIMEN v1.4.0 以上^{*11} をお使いの場合、`rpicam-still --list-cameras` コマンドで利用可能なカメラの一覧を表示可能です:

-
4. <https://gist.github.com/satakagi/2c5be63d4759fd21eca939f507e7f7ef>
 5. <https://www.amazon.co.jp/dp/B073RCXGQS/>
 6. <https://www.amazon.co.jp/dp/B086MK17K5/>
 7. <https://www.amazon.co.jp/dp/B08HVRB59N/>
 8. <https://www.amazon.co.jp/gp/product/B07QH455KY/>
 9. <https://projects.raspberrypi.org/ja-JP/projects/getting-started-with-picamera>
 10. <https://github.com/chirimen-oh/chirimen-lite/releases>
 11. <https://github.com/chirimen-oh/chirimen-lite/releases>

```
$ rpikam-still --list-cameras
Available cameras
-----
0 : ov5647 [2592x1944 10-bit GBRG] (/base/soc/i2c0mux/i2c@1/ov
5647@36)
    Modes: 'SGBRG10_CSI2P' : 640x480 [58.92 fps - (16, 0)/2560
x1920 crop]
                                1296x972 [43.25 fps - (0, 0)/2592
x1944 crop]
                                1920x1080 [30.62 fps - (348, 434)
/1928x1080 crop]
                                2592x1944 [15.63 fps - (0, 0)/259
2x1944 crop]
```

詳細: Camera software - Raspberry Pi Documentation^{*12}

CHIRIMEN ドライバのインストール

- `pi-camera-connect` を使用します。

サンプル

`pi-camera-connect` のリポジトリ^{*13} の readme と同じ内容ですが、オプションを加えてみました

```
import { StillCamera } from "pi-camera-connect";
import * as fs from "fs";

// Take still image and save to disk
```

12. https://www.raspberrypi.com/documentation/computers/camera_software.html

13. <https://github.com/launchcodedev/pi-camera-connect>

```

async function runApp() {
  const stillCamera = new StillCamera({
    width: 600,
    height: 600,
  });
  const image = await stillCamera.takeImage();
  fs.writeFileSync("still-image.jpg", image);
}

runApp();

```

dataURL を取得

```

import { StillCamera } from "pi-camera-connect";

// Take still image and get dataURI
async function runApp() {
  const stillCamera = new StillCamera({
    width: 200,
    height: 200,
  });
  var mime = "image/jpeg";
  var encoding = "base64";
  const image = await stillCamera.takeImage();
  const b64str = image.toString(encoding);
  const dataURL = "data:" + mime + ";" + encoding + "," + b64str;
  console.log(dataURL);
}

runApp();

```

Note

- dataURL^{*14}で画像を文字列化すれば比較的簡単にサーバに送信したりできるでしょう。
 - リモートカメラサンプル^{*15}

- 各種センサ (WebGPIO 経由で人感センサーなど)を使い、自動的に撮影、サーバにアップロードする仕組みなどでもできるでしょう。
- pi-camera-connect のリポジトリ^{*16}
- この章はこちらの記事^{*17}を改変して作成されました。

サンプルコード (main.js)

```
// GPIO5のスイッチを押すと、Raspberry Pi Cameraで撮影し、ファイルに保存する

// ライブラリ pi-camera-connect をまずインストールする必要があります。readmeを参照してください。

import { StillCamera } from "pi-camera-connect";
import * as fs from "fs";

import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js";
const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

const stillCamera = new StillCamera({
  width: 600,
  height: 600,
});

async function switchCheck() {
  const gpioAccess = await requestGPIOAccess();
  const port = gpioAccess.ports.get(5);
```

14. https://developer.mozilla.org/ja/docs/Web/HTTP/Basics_of_HTTP/Data_URIs

15. https://tutorial.chirimen.org/pizero/esm-examples/#REMOTE_remote_camera

16. <https://github.com/launchcodedev/pi-camera-connect>

17. <https://x.gd/i3u0x>

```
await port.export("in");
port.onchange = takeImage;
}

async function takeImage(ev) {
  if (ev.value == 1) {
    // 押し下げたときだけ撮影
    return;
  }
  const image = await stillCamera.takeImage();
  const fileName = "still-image-" + new Date().getTime() + ".jpg";
  console.log("sw:", ev.value, " takeImage:", fileName);
  fs.writeFileSync(fileName, image);
}

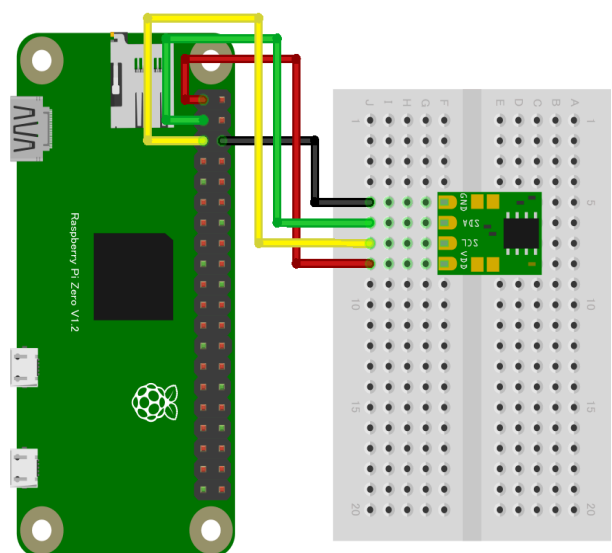
switchCheck();
```

環境センサー

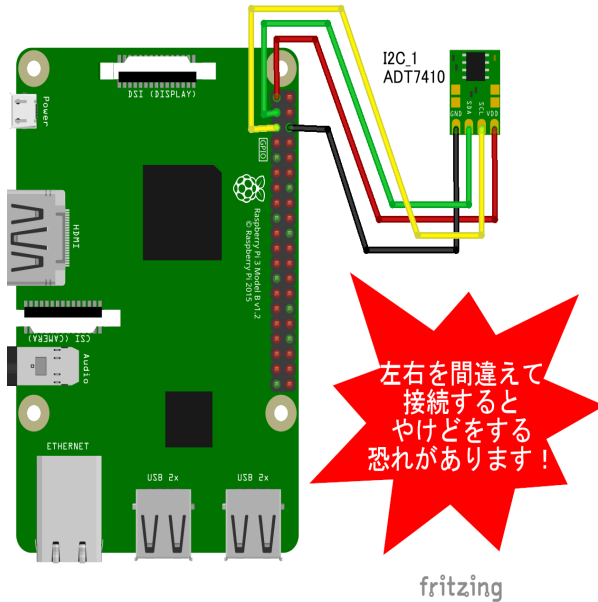
温度・湿度センサー

ADT7410 温度センサー

配線図



fritzing



CHIRIMEN ドライバのインストール

```
npm i @chirimen/adt7410
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import ADT7410 from "@chirimen/adt7410";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess(); // i2cAccessを非同期で取得
  const port = i2cAccess.ports.get(1); // I2C I/Fの1番ポートを取得
  const adt7410 = new ADT7410(port, 0x48); // 取得したポートの0x48ア
```

ドレスをADT7410ドライバで受信する

```
await adt7410.init();
for (;;) {
  // 無限ループ
  const value = await adt7410.read();
  console.log(`${value} degree`);
  await sleep(1000);
}
```

AHT10 温湿度センサー

配線図

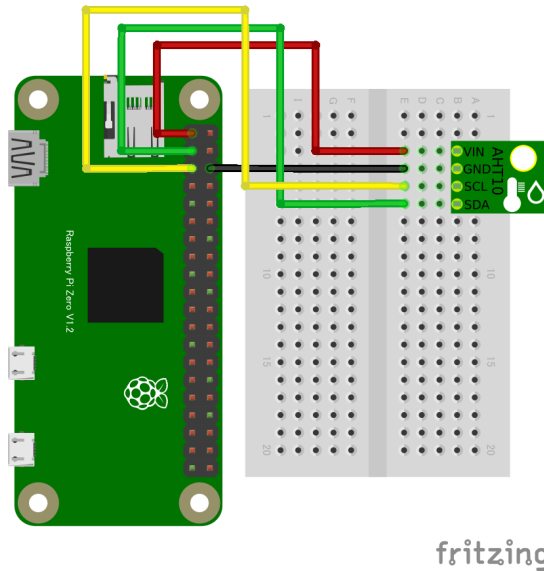


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/aht10
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import AHT10 from "@chirimen/aht10";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const aht10 = new AHT10(port, 0x38);
  await aht10.init();

  while (true) {
    const { humidity, temperature } = await aht10.readData();
    console.log(
      [
        `Humidity: ${humidity.toFixed(2)}%`,
        `Temperature: ${temperature.toFixed(2)} degree`
      ].join(", ")
    );

    await sleep(500);
  }
}
```

AHT20 温湿度センサー

配線図

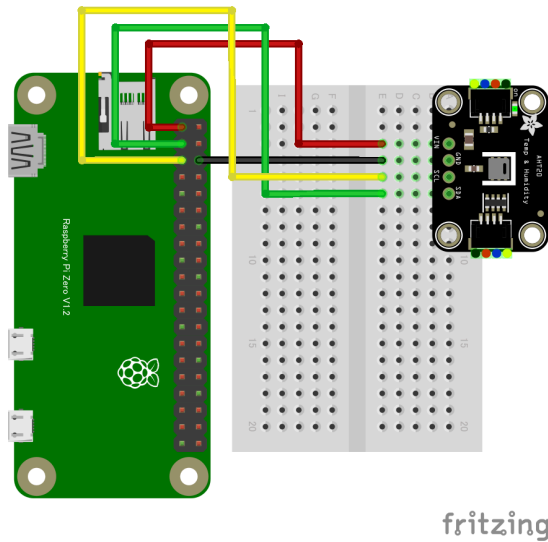


図 1: 配線図

Note: このデバイスで使用するドライバahtx0.jsは、AHT20 及びAHT10 で使用可能です。

CHIRIMEN ドライバのインストール

```
npm i @chirimen/ahtx0
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import AHT20 from "@chirimen/ahtx0";
const sleep = msec => new Promise(resolve => setTimeout(resolve,
```

```
msec));  
  
main();  
  
async function main() {  
  const i2cAccess = await requestI2CAccess();  
  const port = i2cAccess.ports.get(1);  
  const aht20 = new AHT20(port, 0x38);  
  await aht20.init();  
  
  while (true) {  
    const { humidity, temperature } = await aht20.readData();  
    console.log(  
      [  
        `Humidity: ${humidity.toFixed(2)}%`,  
        `Temperature: ${temperature.toFixed(2)} degree`,  
      ].join(", ")  
    );  
  
    await sleep(500);  
  }  
}
```

HTU21D 温湿度センサー

配線図

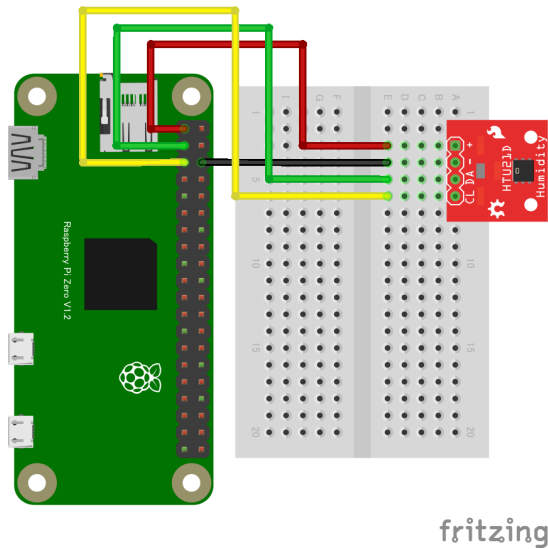


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/htu21d
```

サンプルコード (main.js)

```
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import HTU21D from "@chirimen/htu21d";
const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const htu21d = new HTU21D(port, 0x40);
  await htu21d.init();

  while (true) {
    var temp = await htu21d.readTemperature();
    var humi = await htu21d.readHumidity();
    console.log("Temperature:", temp, " Humidity", humi);
    await sleep(1000);
  }
}
```

SHT30 温湿度センサー

配線図

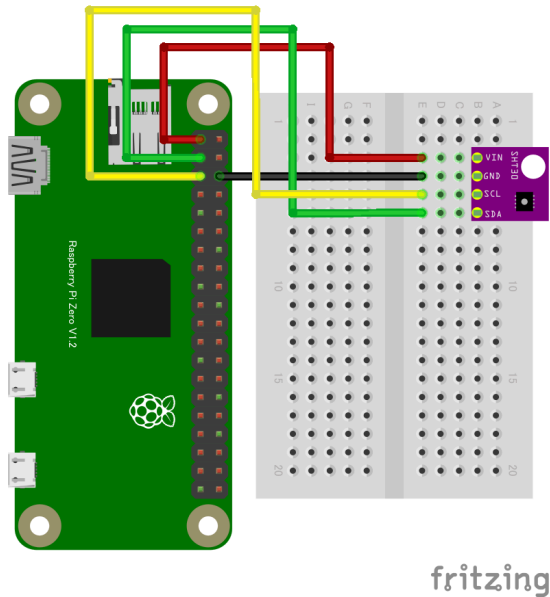


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/sht30
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import SHT30 from "@chirimen/sht30";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const sht30 = new SHT30(port, 0x44);
  await sht30.init();

  while (true) {
    const { humidity, temperature } = await sht30.readData();
    console.log(
      [
        `Humidity: ${humidity.toFixed(2)}%`,
        `Temperature: ${temperature.toFixed(2)} degree`
      ].join(", ")
    );

    await sleep(500);
  }
}
```

SHT40 温湿度センサー

配線図

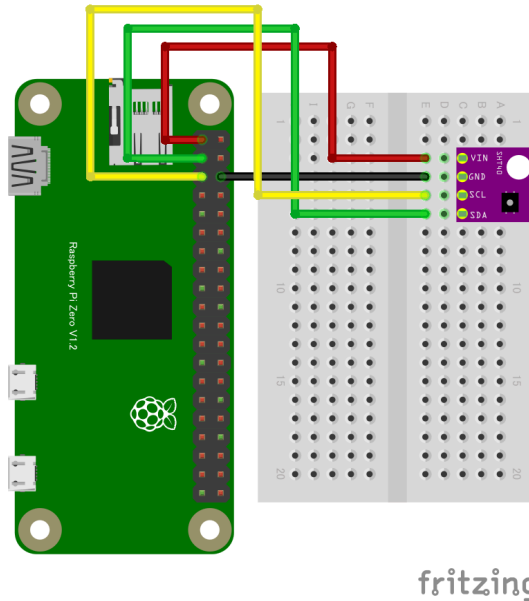


図1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/sht40
```

サンプルコード (main.js)

```
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import SHT40 from "@chirimen/sht40";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const sht40 = new SHT40(port, 0x44);
  await sht40.init();

  while (true) {
    const { humidity, temperature } = await sht40.readData();
    console.log(
      [
        `Humidity: ${humidity.toFixed(2)}%`,
        `Temperature: ${temperature.toFixed(2)} degree`
      ].join(", ")
    );

    await sleep(500);
  }
}
```

気圧センサー

BMP180 大気圧温度センサー

配線図

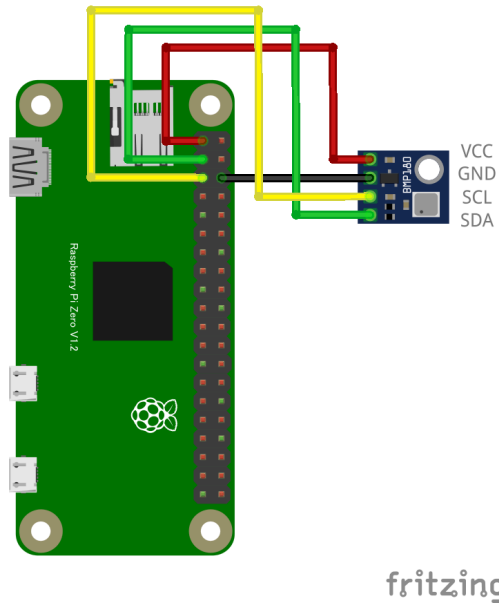


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/bmp180
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";  
import BMP180 from "@chirimen/bmp180";
```

```
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const readInterval = 1000;

  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const bmp180 = new BMP180(port, 0x77);
  await bmp180.init();
  for (;;) {
    const pressure = await bmp180.readPressure();
    const temperature = await bmp180.readTemperature();
    console.log(
      `Pressure: ${pressure.toFixed(2)} Pa, Temperature: ${temperature} degree.`
    );
    await sleep(readInterval);
  }
}
```

BMP280 温度・気圧センサー

配線図

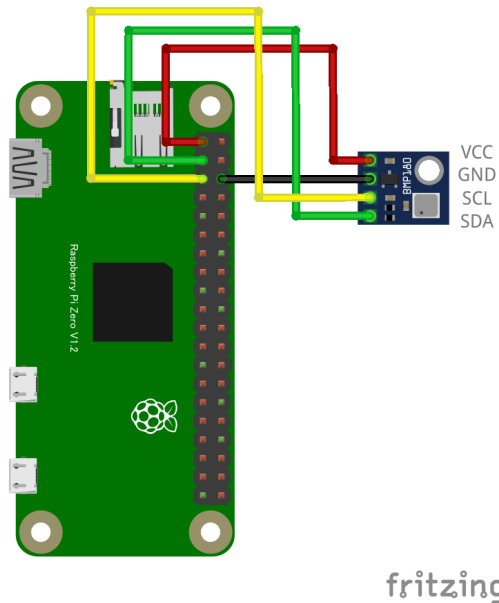


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/bmp280
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import BMP280 from "@chirimen/bmp280";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const bmp280 = new BMP280(port, 0x76);
  await bmp280.init();

  while (true) {
    const data = await bmp280.readData();
    const pressure = data.pressure.toFixed(2);
    const temperature = data.temperature.toFixed(2);
    console.log(
      [`Temperature: ${temperature} degree`, `Pressure: ${pressure} hPa`].join(
        ", "
      )
    );

    await sleep(500);
  }
}
```

BME280 温度・湿度・気圧センサー

配線図

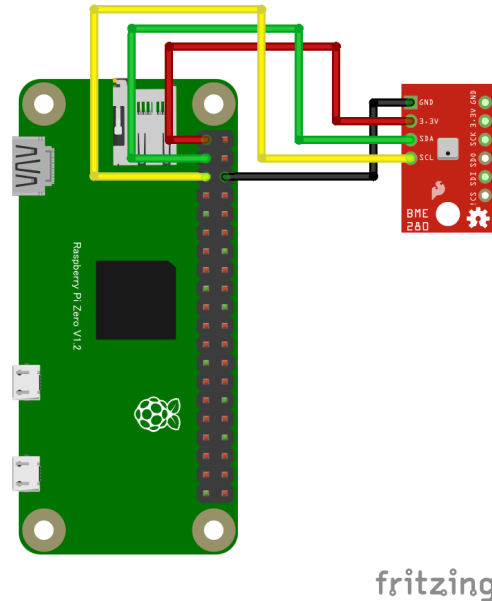


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/bme280
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import BME280 from "@chirimen/bme280";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const bme280 = new BME280(port, 0x76);
  await bme280.init();

  while (true) {
    const data = await bme280.readData();
    const temperature = data.temperature.toFixed(2);
    const humidity = data.humidity.toFixed(2);
    const pressure = data.pressure.toFixed(2);
    console.log(
      [
        `Temperature: ${temperature} degree`,
        `Humidity: ${humidity} %`,
        `Pressure: ${pressure} hPa`
      ].join(", ")
    );
    await sleep(500);
  }
}
```

空気品質センサー

ENS160 空気質センサ

配線図

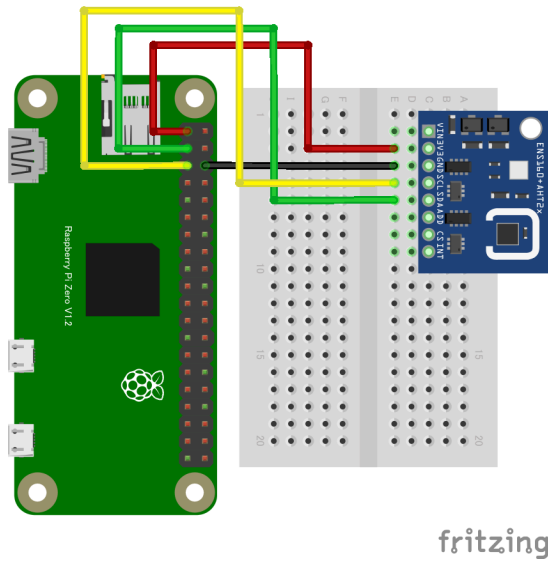


図 1: 配線図

Note: 配線図ののモジュールは、ENS160 に加えて AHT20(温度湿度センサ)も載っている複合センサボードです。(ENS160 は温度と湿度の設定が必要) Amazon の商品例^{*1}

CHIRIMEN ドライバのインストール

```
npm i @chirimen/ens160
```

1. <https://www.amazon.co.jp/dp/B0D41R4V3Z>

サンプルコード (main.js)

```

import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import ENS160 from "@chirimen/ens160";
const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const ens160 = new ENS160(port, 0x53);
  await ens160.init();

  await ens160.set_temperature(22); // 温度を設定する(℃)
  await ens160.set_humidity(52); // 湿度を設定する(%)
  await sleep(1500);
  console.log(
    `setT:${(await ens160.get_temperature()).toFixed(2)} [deg`
    + `.] setH:${(await ens160.get_humidity()).toFixed(2)} [%]`
  );

  while (true) {
    const { AQI, TVOC, eCO2 } = await ens160.get_data();
    const mode = await ens160.get_mode();
    console.log(
      [
        `AQI(1-5):${AQI}`,
        `TVOC:${TVOC}ppb`,
        `eCO2:${eCO2}ppm`,
        `mode:${mode}(0:sleep,1:idle,2:standard)`,
      ].join(", ")
    );
  }
}

```

```
        await sleep(500);  
    }  
}
```

SGP40 ガスセンサー

配線図

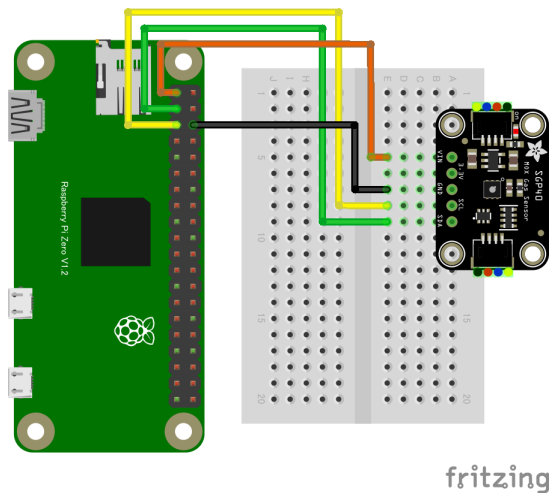


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/sgp40
```

サンプルコード (main.js)

```
/* 各種ライブラリをインポート */  
import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js"; // WebGPIO  
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js"; // WebI2C  
import SGP40 from "@chirimen/sgp40"; // SGP40
```

```

const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

main();

var sgp40;
async function main() {
  const i2cAccess = await requestI2CAccess();
  const i2cPort = i2cAccess.ports.get(1);
  sgp40 = new SGP40(i2cPort);
  await sgp40.init();
  while (true) {
    // var sgp = await sgp40.raw(); // 温度・湿度 内部で決め打ち
    // (25℃, 50%)
    var sgp = await sgp40.measureRaw(25, 50); // 温度・湿度決め
    打ち
    console.log("Gas:", sgp );
    await sleep(1000);
  }
}

```

CO2 センサー

SCD40 CO2センサー

配線図

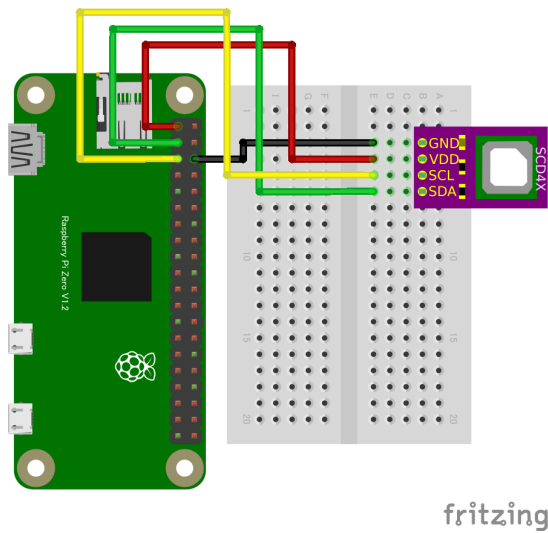


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/scd40
```

サンプルコード (main.js)

```
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";  
import SCD40 from "@chirimen/scd40";
```

```
const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const scd40 = new SCD40(port, 0x62);
  await scd40.init();
  console.log(await scd40.serial_number());
  await scd40.start_periodic_measurement();

  // 値が出てくるまで数秒かかります
  // 測定が更新されると、updatedフラグがtrueになります
  while (true) {
    var data = await scd40.getData();
    console.log(data);
    await sleep(1000);
  }
}
```

光センサー

BH1750 光センサー

配線図

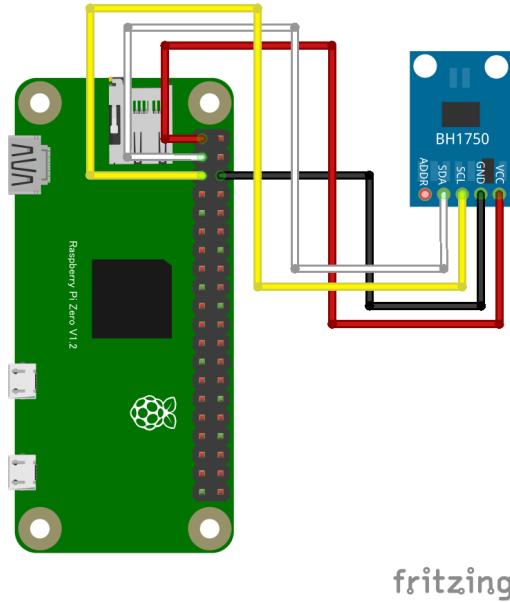


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/bh1750
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index
.js";
import BH1750 from "@chirimen/bh1750";
```

```
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const bh1750 = new BH1750(port, 0x23);
  await bh1750.init();

  while (true) {
    const lux = await bh1750.measure_high_res();
    console.log(lux.toFixed(3) + "lx");

    await sleep(500);
  }
}
```

LTR390 UVセンサー

配線図

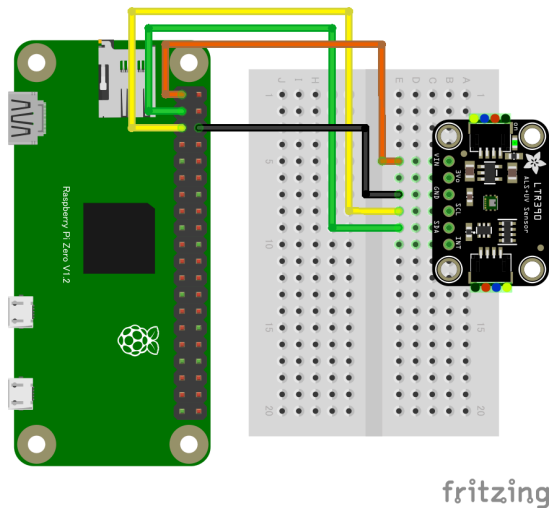


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/ltr390
```

サンプルコード (main.js)

```
/* 各種ライブラリをインポート */
import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js"; // WebGPIO
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js"; // WebI2C
import LTR390 from "@chirimen/ltr390"; // LTR390
```

```
const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

main();

var ltr390;
async function main() {
  /* センサー初期化 */
  const i2cAccess = await requestI2CAccess();
  const ic2Port = i2cAccess.ports.get(1);
  ltr390 = new LTR390(ic2Port);
  await ltr390.init();
  while (true) {
    var UVS = await ltr390.UVS();
    console.log(UVS, " UVS");
    await sleep(1000);
  }
}
```

TSL2561 Grove Digital Light Sensor 光センサー

配線図

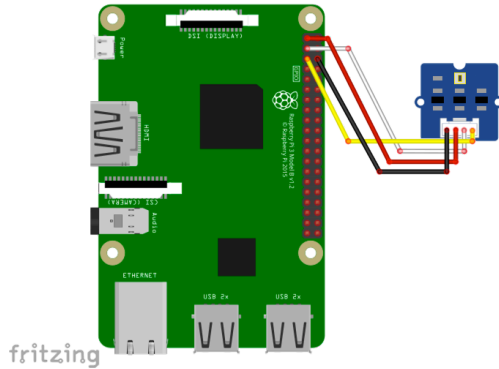


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/grove-light
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "./node_modules/node-web-i2c/index.js";
import GROVELIGHT from "@chirimen/grove-light";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const grovelight = new GROVELIGHT(port, 0x29);
  await grovelight.init();
  for (;;) {
    try {
      const value = await grovelight.read();
      console.log(value);
    } catch (error) {
      console.error(" Error : ", error);
    }
    await sleep(200);
  }
}
```

TSL2591照度センサー

配線図

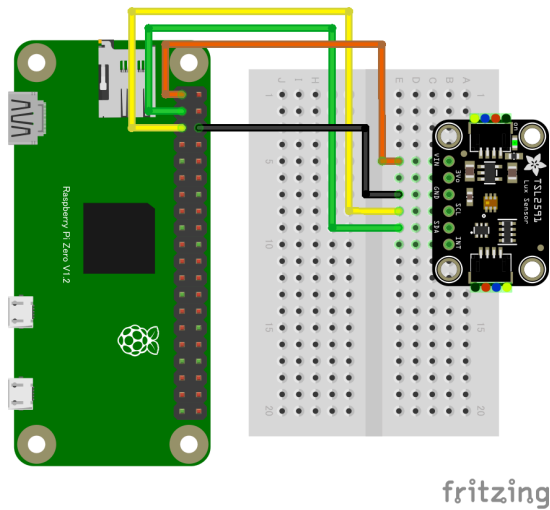


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/tsl2591
```

サンプルコード (main.js)

```
/* 各種ライブラリをインポート */  
import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js"; // WebGPIO  
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js"; // WebI2C  
import TSL2591 from "@chirimen/tsl2591"; // TSL2591
```

```
const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

main();

var tsl2591;
async function main() {
  /* センサー初期化 */
  const i2cAccess = await requestI2CAccess();
  const ic2Port = i2cAccess.ports.get(1);
  tsl2591 = new TSL2591(ic2Port);
  await tsl2591.init();
  while (true) {
    var lux = await tsl2591.Lux();
    console.log(lux , " Lux");
    await sleep(1000);
  }
}
```

VEML6070 UV センサー

配線図

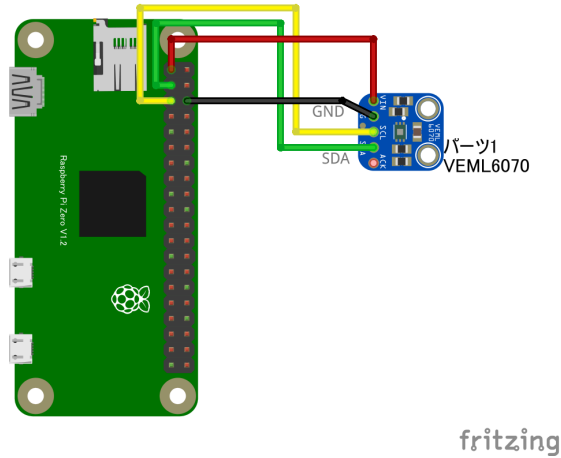


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/veml6070
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "./node_modules/node-web-i2c/index.js";
import VEML6070 from "@chirimen/veml6070";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

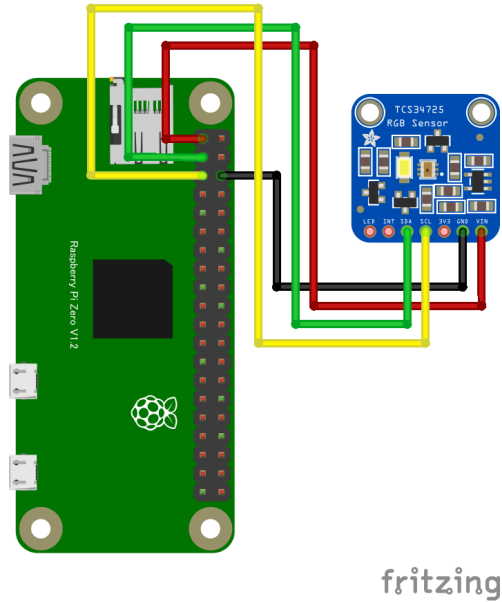
async function main() {
```

```
const i2cAccess = await requestI2CAccess();
const port = i2cAccess.ports.get(1);
const veml6070 = new VEML6070(port);
await veml6070.init();
for (;;) {
  const value = await veml6070.read();
  console.log(value);
  await sleep(200);
}
}
```

色センサー

TCS34725 カラーセンサー

配線図



fritzing

図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/tcs34725
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import TCS34725 from "@chirimen/tcs34725";
```

```
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const tcs34725 = new TCS34725(port, 0x29);
  await tcs34725.init();

  // You can select the value of gain from 1, 4, 16 or 60.
  await tcs34725.gain(4);

  while (true) {
    const data = await tcs34725.read();
    console.log(
      [
        `R: ${data.r}`,
        `G: ${data.g}`,
        `B: ${data.b}`,
        `Clear Light: ${data.c}`
      ].join(", ")
    );

    await sleep(500);
  }
}
```

S11059 デジタルカラーセンサー

配線図

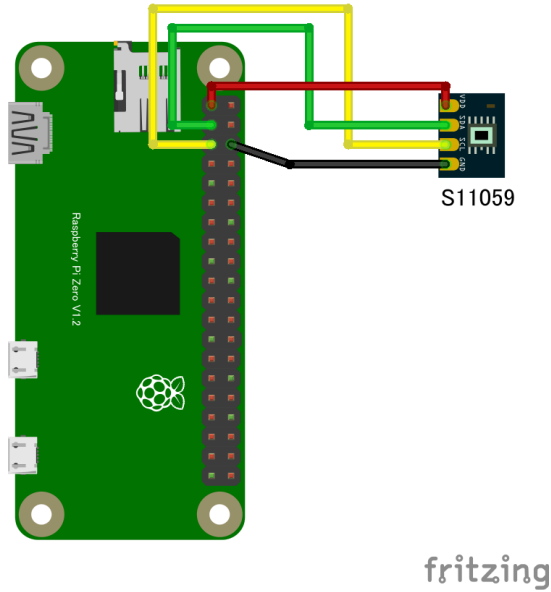


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/s11059
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import S11059 from "@chirimen/s11059";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const s11059 = new S11059(port, 0x2a);
  await s11059.init();
  for (;;) {
    try {
      const values = await s11059.readR8G8B8();
      const red = values[0] & 0xff;
      const green = values[1] & 0xff;
      const blue = values[2] & 0xff;
      const gain_level = values[3];
      console.log(`R:${red} G:${green} B:${blue} GAIN: ${gain_level}`);
    } catch (error) {
      console.error("READ ERROR:" + error);
    }
    await sleep(1000);
  }
}
```

モーション・位置センサー

加速度・ジャイロセンサー

ADXL345 Grove Accelerometer 3 軸加速度センサー

配線図

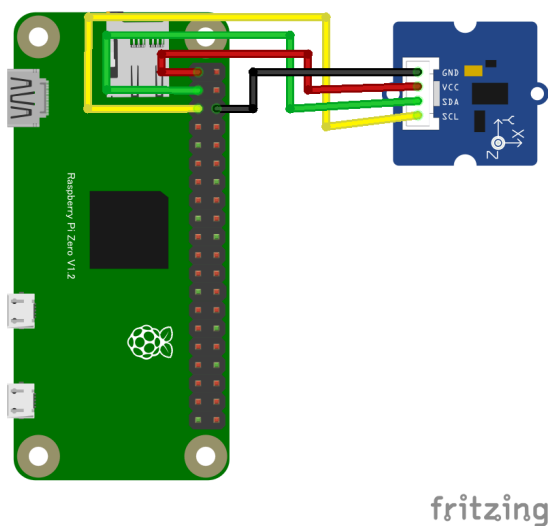


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/grove-accelerometer
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index
.js";
import GROVEACCELEROMETER from "@chirimen/grove-accelerometer";
const sleep = msec => new Promise(resolve => setTimeout(resolve,
msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const groveaccelerometer = new GROVEACCELEROMETER(port, 0x53);
  await groveaccelerometer.init();
  for (;;) {
    try {
      const values = await groveaccelerometer.read();
      console.log(`ax: ${values.x}, ay: ${values.y}, az: ${values
.z}`)
    } catch (err) {
      console.error("READ ERROR:" + err);
    }
    await sleep(1000);
  }
}
```

ICM20948 9軸センサー

配線図

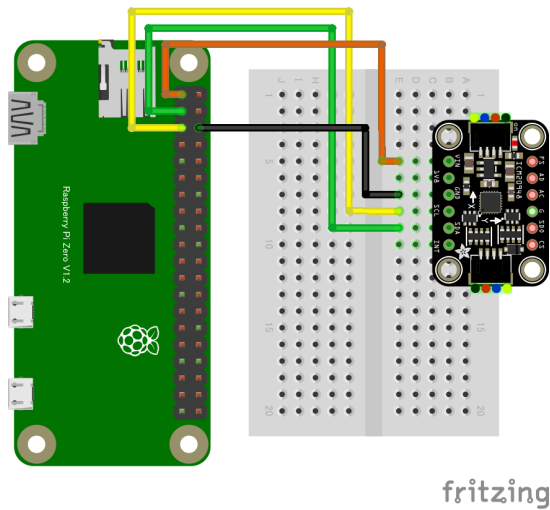


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/icm20948
```

サンプルコード (main.js)

```
/* 各種ライブラリをインポート */
import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js"; // WebGPIO
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js"; // WebI2C
import ICM20948 from "@chirimen/icm20948"; // ICM20948
```

```

const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

main();

var icm20948;
async function main() {
  /* センサー初期化 */
  const i2cAccess = await requestI2CAccess();
  const ic2Port = i2cAccess.ports.get(1);
  icm20948 = new ICM20948(ic2Port);
  await icm20948.init();
  while (true) {
    var icm = await icm20948.getdata();
    console.log("/-----
-----/");
    console.log(`Roll = ${icm[0].toFixed(2)} , Pitch = ${icm[1].toFixed(2)} , Yaw = ${icm[2].toFixed(2)}`);
    console.log(`Acceleration: X = ${icm[3]}, Y = ${icm[4]}, Z = ${icm[5]}`);
    console.log(`Gyroscope:      X = ${icm[6]} , Y = ${icm[7]} , Z = ${icm[8]}`);
    console.log(`Magnetic:      X = ${icm[9]} , Y = ${icm[10]} , Z = ${icm[11]}`);
    await sleep(1000);
  }
}

```

MPU6050 3 軸ジャイロ 3 軸加速度 複合センサー

回路図

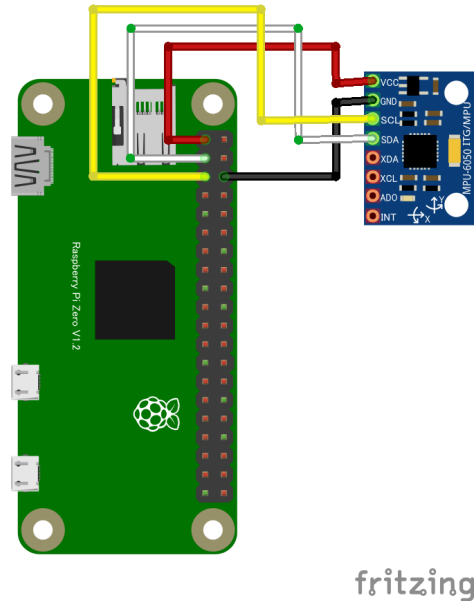


図 1: 回路図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/mpu6050
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index
.js";
import MPU6050 from "@chirimen/mpu6050";
const sleep = msec => new Promise(resolve => setTimeout(resolve,
msec));
```

```
main();

async function main() {
  var i2cAccess = await requestI2CAccess();
  var port = i2cAccess.ports.get(1);
  var mpu6050 = new MPU6050(port, 0x68);
  await mpu6050.init();
  while (true) {
    const data = await mpu6050.readAll();
    const temperature = data.temperature.toFixed(2);
    const g = [data.gx, data.gy, data.gz];
    const r = [data.rx, data.ry, data.rz];
    console.log(
      [
        `Temperature: ${temperature} degree`,
        `Gx: ${g[0]}, Gy: ${g[1]}, Gz: ${g[2]}`,
        `Rx: ${r[0]}, Ry: ${r[1]}, Rz: ${r[2]}`
      ].join("\n")
    );

    await sleep(500);
  }
}
```

MPU9250 3 軸ジャイロ 3 軸加速度 3 軸 磁気複合センサー

配線図

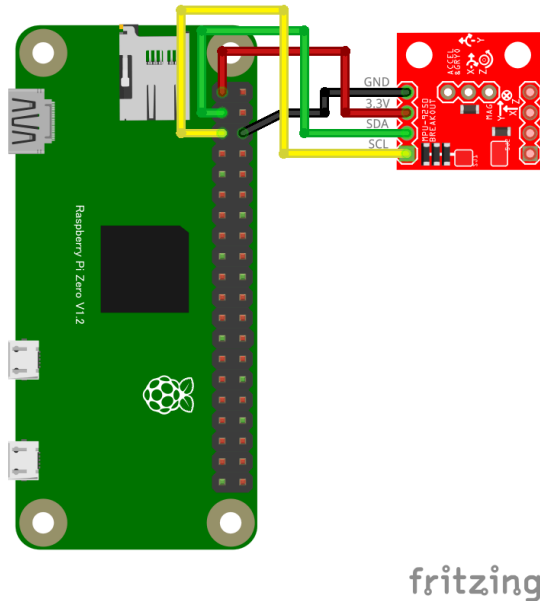


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/ak8963
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import MPU6500 from "@chirimen/mpu6500";
import AK8963 from "@chirimen/ak8963";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const mpu6500 = new MPU6500(port, 0x68);
  const ak8963 = new AK8963(port, 0x0c);
  await mpu6500.init();
  await ak8963.init();

  while (true) {
    const g = await mpu6500.getGyro();
    const r = await mpu6500.getAcceleration();
    const h = await ak8963.readData();
    console.log(
      [
        `Gx: ${g.x}, Gy: ${g.y}, Gz: ${g.z}`,
        `Rx: ${r.x}, Ry: ${r.y}, Rz: ${r.z}`,
        `Hx: ${h.x}, Hy: ${h.y}, Hz: ${h.z}`
      ].join("\n")
    );

    await sleep(500);
  }
}
```

距離センサー

GP2Y0E03 測距センサー 40 mm - 0.1 m

配線図

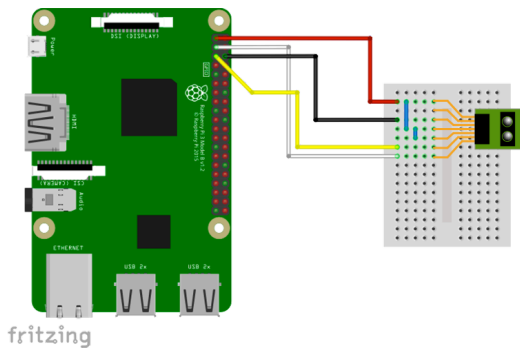


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/gp2y0e03
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "node-web-i2c";
import GP2Y0E03 from "@chirimen/gp2y0e03";
const sleep = msec => new Promise(resolve => setTimeout(resolve,
```

```
msec));

main();

async function main() {
  try {
    const i2cAccess = await requestI2CAccess();
    const port = i2cAccess.ports.get(1);
    const sensor_unit = new GP2Y0E03(port, 0x40);
    await sensor_unit.init();

    while (1) {
      try {
        const distance = await sensor_unit.read();
        if (distance != null) {
          console.log("Distance:" + distance + "cm");
        } else {
          console.log("out of range");
        }
      } catch (err) {
        console.error("READ ERROR:" + err);
      }
      await sleep(500);
    }
  } catch (err) {
    console.error("GP2Y0E03 init error");
  }
}
```

VL53L0X レーザー測距センサー 30 mm - 2 m

配線図

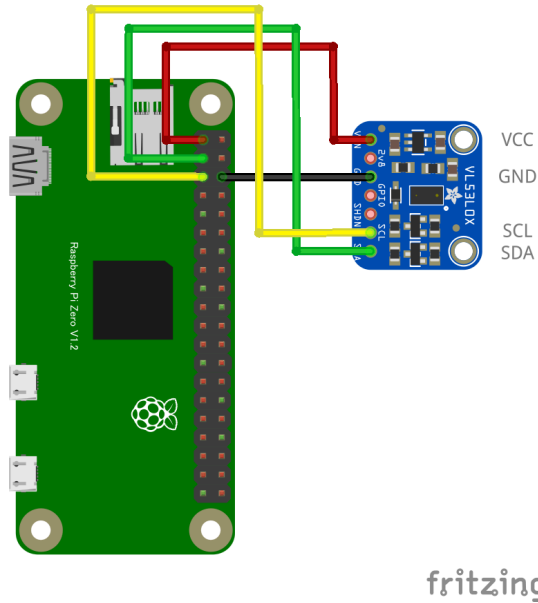


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/vl53l0x
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import VL53L0X from "@chirimen/vl53l0x";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const vl = new VL53L0X(port, 0x29);
  await vl.init(); // for Long Range Mode (<2m) : await vl.init(true);
  for (;;) {
    const distance = await vl.getRange();
    console.log(`${distance} [mm]`);
    await sleep(500);
  }
}
```

VL53L1X レーザー距離センサー

配線図

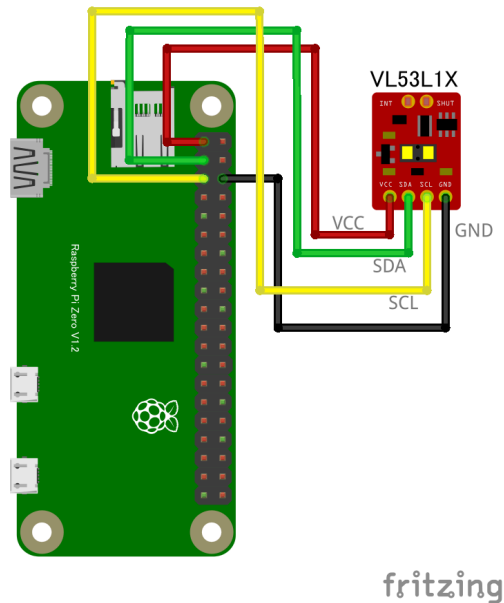


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/vl53l1x
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import VL53L1X from "@chirimen/vl53l1x";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const vl53l1x = new VL53L1X(port, 0x29);

  // Mode: short, medium, long
  await vl53l1x.init("short");

  // Necessary to start measurement
  await vl53l1x.startContinuous();

  while (true) {
    const distance = await vl53l1x.read();
    console.log(distance.toFixed(2) + " mm");
    await sleep(500);
  }
}
```

角度センサー

AS5600 磁気式角度センサ

ブレークアウトボードを購入すると付属している円盤状の磁石をチップに近接配置。この磁石の回転角を非接触で検出できるセンサです

- 円盤状磁石の中心を回転軸とし、この角度を検出
- チップの中心と、円盤状磁石の中心を一致させる
- チップ表面と磁石の距離は0.5～3mm程度にする (参照情報(データシート 33 ページ)^{*1})
- DIR ピンを VCC に接続すると角度の方向が逆向きになる
- ブレークアウトボード
 - <https://electronicwork.shop/items/64205dc6cd92fe0096fb7d5c>^{*2}
 - <https://www.switch-science.com/products/3493>^{*3}
- ソフトウェア
 - as5600.js^{*4} : ドライバ
 - main.js : サンプルアプリ
 - 両方とも myApp ディレクトリ直下に置く
- AS5600 データシート

-
1. https://ams.com/documents/20143/36005/AS5600_DS000365_5-00.pdf#page=34
 2. <https://electronicwork.shop/items/64205dc6cd92fe0096fb7d5c>
 3. <https://www.switch-science.com/products/3493>
 4. <https://raw.githubusercontent.com/chirimen-oh/chirimen-drivers/master/packages/as5600/as5600.js>

- https://ams.com/documents/20143/36005/AS5600_DS000365_5-00.pdf^{*5}

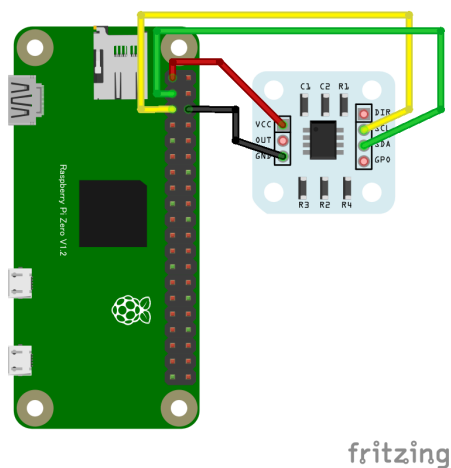


図 1: AS5600.png

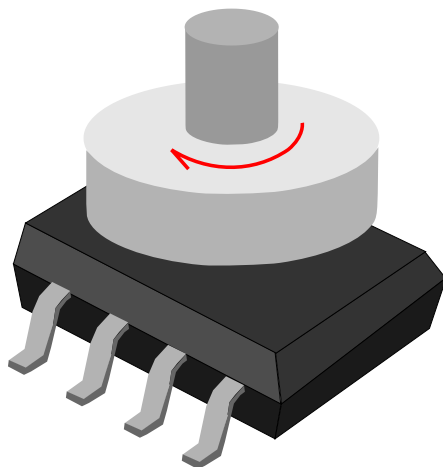


図 2: AS5600_drawing.svg

5. https://ams.com/documents/20143/36005/AS5600_DS000365_5-00.pdf

CHIRIMEN ドライバのインストール

- `as5600.js` を使用します。

サンプルコード (main.js)

```
// AS5600(磁気式角度センサ)を使う
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import AS5600 from "../as5600.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

var as5600;

async function main() {
    const i2cAccess = await requestI2CAccess();
    const port = i2cAccess.ports.get(1);
    as5600 = new AS5600(port);
    await as5600.init();
    var stat = await as5600.getStatus();
    console.log("stat:" , stat);
    startMeasurement();
}

async function startMeasurement() {
    for (; ;) {
        var rawAngle = await as5600.getAngle();
        console.log("rawA:", rawAngle);
        await sleep(1000);
    }
}
```

ジェスチャーセンサー

PAJ7620 Grove Gesture ジェスチャー認識センサー

配線図

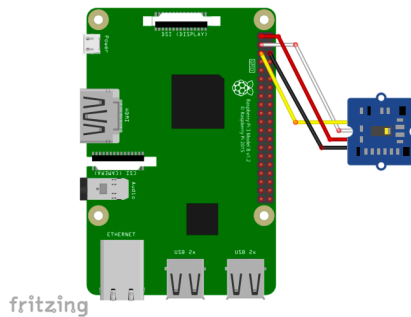


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/grove-gesture
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import PAJ7620 from "@chirimen/grove-gesture";
```

```
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const gesture = new PAJ7620(port, 0x73);
  await gesture.init();

  for (;;) {
    const v = await gesture.read();
    console.log(v);
    await sleep(1000);
  }
}
```

雷センサー

AS3935 雷センサー

概要

AS3935は、雷センサーICです。ICに内蔵された雷検出アルゴリズムが入力信号をチェックし暴風雨の頭部までの距離を推定します。

このサンプルでは、I2Cバスに加えて、GPIOポート5を雷検出のトリガーとして使用しています。

特記事項

しっかりと動作確認ができていません。(雷検出の機会が少ないため) 動作確認にご協力ください。(ISSUEにレポートを記載いただけると幸いです)

使用するブレイクアウトボード

- I2Cスレーブアドレス、`0x03` が設定されているものを使用しています。
- 秋月電子通商 AE-AS3935 がこれに相当します。: 製品ページ^{*1}

I2Cデバイスの検出

- I2Cアドレス `0x03` のデバイスは、通常の `i2cdetect` コマンドでは検出できません(標準範囲外アドレス) `-a` オプションが必要です。コマンドプロンプトから、以下のコマンドで検出してください。
- `i2cdetect -y -a 1`

1. <https://akizukidenshi.com/catalog/g/gK-08685/>

配線図

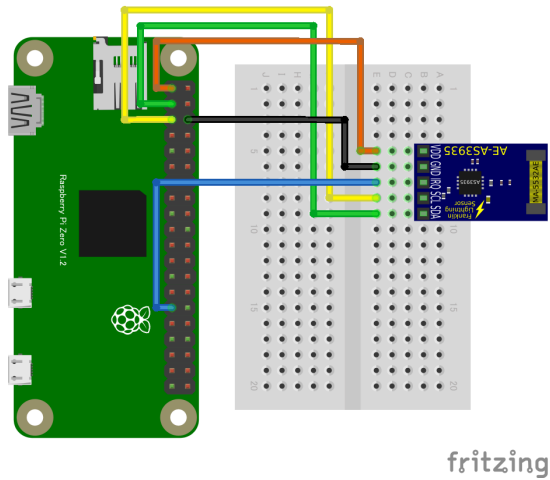


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/as3935
```

サンプルコード (main.js)

```
/* 各種ライブラリをインポート */  
import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js"; // WebGPIO  
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js"; // WebI2C  
import AS3935 from "@chirimen/as3935"; // AS3935  
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```

main();

var as3935;
async function main() {
  /* 雷センサー初期化 */
  const i2cAccess = await requestI2CAccess();
  const ic2Port = i2cAccess.ports.get(1);
  as3935 = new AS3935(ic2Port);
  await as3935.init();
  await as3935.reset();
  await sleep(100);
  await as3935.set_indoors(true);
  await as3935.set_noise_floor(0);
  await as3935.calibrate(0x0f);

  /* 雷センサーインタラプト用GPIO初期化 */
  const gpioAccess = await requestGPIOAccess();
  const as3935_interruptPort = gpioAccess.ports.get(5);
  await as3935_interruptPort.export("in");
  as3935_interruptPort.onchange = as3935_sens;
}

async function as3935_sens(ev) {
  if (ev.value == 0) { return }
  console.log("interruptPort:", ev.value);
  await sleep(10);
  var reason = await as3935.get_interrupt();
  if (reason == 0x01) {
    console.log("Noise level too high - adjusting");
    await as3935.raise_noise_floor();
  } else if (reason == 0x04) {
    console.log("Disturber detected - masking");
    await as3935.set_mask_disturber(true);
  } else if (reason == 0x08) {
    var now = new Date().toLocaleString();
    var distance = await as3935.get_distance();
  }
}

```

```
        console.log(`Detect lightinig! It was ${distance} Km away  
        . (${now}) `);  
    }  
} ``
```

ディスプレイ・LED

OLED ディスプレイ

SSD1306 OLED ディスプレイ

配線図

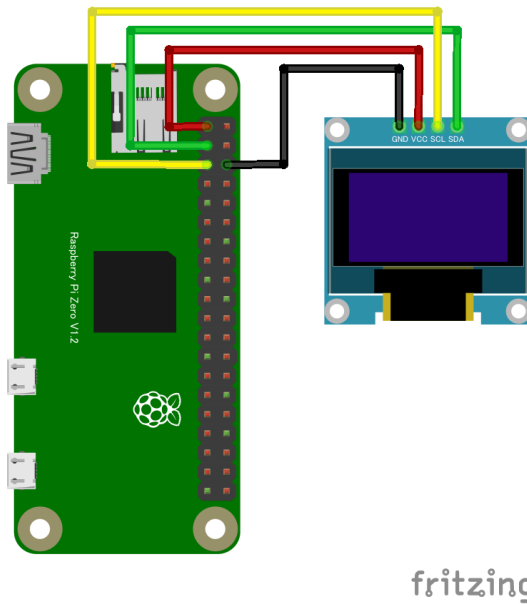


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/grove-oled-display
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import OledDisplay from "@chirimen/grove-oled-display";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  console.log("initializing...");
  const port = i2cAccess.ports.get(1);
  const display = new OledDisplay(port);
  await display.init(true); // SSD1306(grove-oled-display)の場合はtrueを設定(SSD1308の場合はパラメータ不要)
  display.clearDisplayQ();
  await display.playSequence();
  console.log("drawing text...");
  display.drawStringQ(0, 0, "Hello");
  display.drawStringQ(1, 0, "Real");
  display.drawStringQ(2, 0, "World");
  await display.playSequence();
  console.log("completed");
}
```

SSD1308 Grove OLED ディスプレイ

配線図

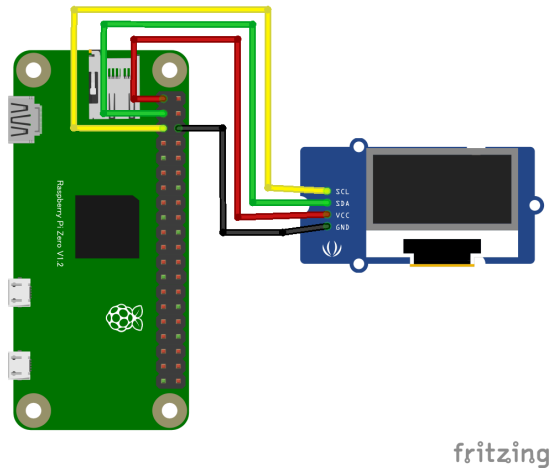


図1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/grove-oled-display
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import OledDisplay from "@chirimen/grove-oled-display";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  console.log("initializing...");
  const port = i2cAccess.ports.get(1);
  const display = new OledDisplay(port);
  await display.init(); // SSD1308の場合はパラメータ不要(SSD1306(grove-oled-display)の場合はtrueを設定)
  display.clearDisplayQ();
  await display.playSequence();
  console.log("drawing text...");
  display.drawStringQ(0, 0, "Hello");
  display.drawStringQ(1, 0, "Real");
  display.drawStringQ(2, 0, "World");
  await display.playSequence();
  console.log("completed");
}
```

LED ドライバー

HT16K33 LEDマトリクスドライバ

HT16K33 LED マトリクスドライバ、8x8LED マトリクスモジュール一体型や7セグメントLED 一体型等にも対応

配線図

Note: 3.3V 電源でも、あまり明るくはなりません動作するようです。

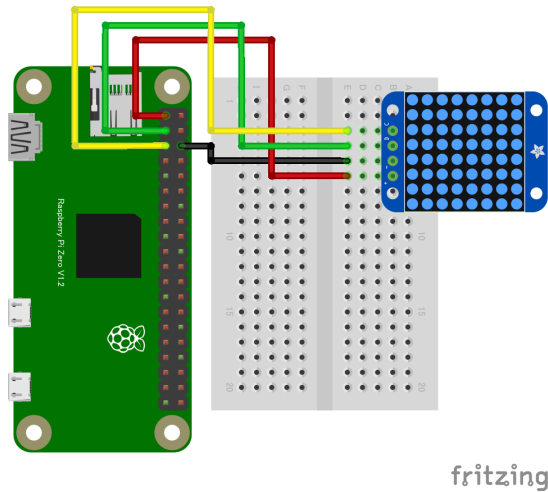


図 1: 配線図

参照情報

8x8 マトリクスLED モジュール(デフォルト設定)

いずれでも動作(ピン配置は異なる)

- Adafruit Mini 8x8 LED Matrix w/I2C Backpack
 - <https://www.switch-science.com/products/1493>^{*1}
 - <https://www.adafruit.com/product/870>^{*2}
- keystudio Ks0064 I2C 8x8 LED Matrix HT16K33
 - <https://ja.aliexpress.com/item/32886174149.html>^{*3}
- サンプルコード [./main.js](#)

Aitendo 製 8x8 マトリクス LED モジュール

Adafruit、keystudio とマトリクスの結線が異なりますが、設定変更により使用可能です。

- 対応品リスト
 - <https://www.aitendo.com/product/12822>^{*4}
 - <https://www.aitendo.com/product/12850>^{*5}
 - <https://www.aitendo.com/product/12823>^{*6}
- サンプルコード [./main-ht16k33_8x8aitendo.js](#)
 - aitendo 製モジュール用の設定変更関数を呼び出して使います。

4桁7セグメント LED モジュール

配線が同じであれば、amazon や aliexpress 等で販売されているジェネリック品でも使用できると思います。(動作は Aitendo 製品で確認)

-
1. <https://www.switch-science.com/products/1493>
 2. <https://www.adafruit.com/product/870>
 3. <https://ja.aliexpress.com/item/32886174149.html>
 4. <https://www.aitendo.com/product/12822>
 5. <https://www.aitendo.com/product/12850>
 6. <https://www.aitendo.com/product/12823>

- 対応品：aitendo
 - <https://www.aitendo.com/product/14540>^{*7}
 - <https://www.aitendo.com/product/18690>^{*8}
 - <https://www.aitendo.com/product/20785>^{*9}
- サンプルコード [./main-ht16k33_7seg.js](#)

4桁 14セグメントLEDモジュール

数字やアルファベットが表示可能なLEDモジュール。配線が同じであれば、amazonやaliexpress等で販売されているジェネリック品でも使用できると思います。（動作はAitendo製品で確認）

- 対応品：aitendo
 - <https://www.aitendo.com/product/20812>^{*10}
 - <https://www.aitendo.com/product/20811>^{*11}
- サンプルコード [./main-ht16k33_14seg.js](#)

その他

- set_led関数を使用することで、任意のLEDマトリクスに対応できます

CHIRIMEN ドライバのインストール

```
npm i @chirimen/ht16k33
```

-
7. <https://www.aitendo.com/product/14540>
 8. <https://www.aitendo.com/product/18690>
 9. <https://www.aitendo.com/product/20785>
 10. <https://www.aitendo.com/product/20812>
 11. <https://www.aitendo.com/product/20811>

サンプルコード (main.js)

```
// Adafruit Mini 8x8 LED Matrix w/I2C Backpack
// https://www.switch-science.com/products/1493 , https://www.adafruit.com/product/870
// or
// Ks0064 keyestudio I2C 8x8 LED Matrix HT16K33
// https://ja.aliexpress.com/item/32886174149.html

import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import HT16K33 from "@chirimen/ht16k33";
const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

main();

const iconPattern =[
0,0,1,1,1,1,0,0,
0,1,0,0,0,0,1,0,
1,0,1,0,0,1,0,1,
1,0,0,0,0,0,0,1,
1,0,1,0,0,1,0,1,
1,0,0,1,1,0,0,1,
0,1,0,0,0,0,1,0,
0,0,1,1,1,1,0,0,
]; // スマイルマーク

const iconPattern2 =[
1,0,0,0,0,0,0,1,
0,1,0,0,0,0,1,0,
0,0,1,1,1,1,0,0,
0,1,1,1,1,1,1,0,
1,1,0,1,1,0,1,1,
0,1,1,1,1,1,1,0,
0,0,1,0,0,1,0,0,
```

```

1,1,0,0,0,0,1,1
]; // インベーター

const iconPattern3 =[
0,1,0,0,0,0,1,0,
1,1,1,1,1,1,1,0,
1,0,0,1,0,0,1,0,
1,1,0,1,1,0,1,0,
1,0,0,1,0,0,1,0,
1,1,1,1,1,1,1,0,
0,1,1,1,1,1,1,1,
0,1,0,1,0,1,0,1
]; // 犬

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const ht = new HT16K33(port);
  await ht.init();

  //await ht.set_blink(ht.HT16K33_BLINK_1HZ);
  //await ht.set_brightness(6);

  while(true){
    ht.set_8x8_array(iconPattern);
    await ht.write_display();
    await sleep(1000);

    ht.set_8x8_array(iconPattern2);
    await ht.write_display();
    await sleep(1000);

    ht.set_8x8_array(iconPattern3);
    await ht.write_display();
    await sleep(1000);

    /** LEDを一個ずつ設定する関数の使用例

```

```
        for ( var i = 0 ; i < 128 ; i++ ){
            ht.set_led(i, 1);
        }
        await ht.write_display();
        await sleep(1000);
        **/
    }
}
```

NeoPixel

Neopixel LED

配線図

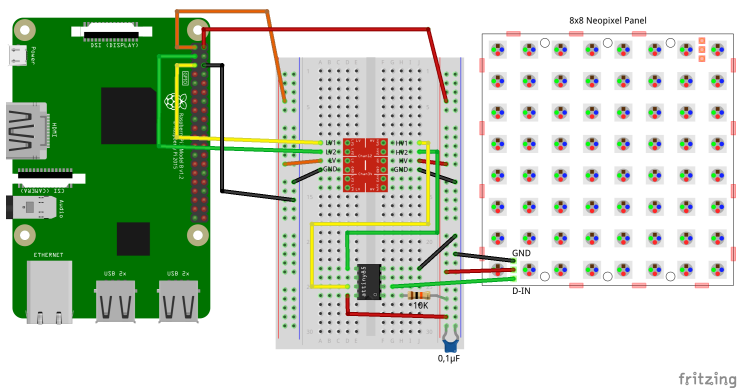


図 1: 配線図

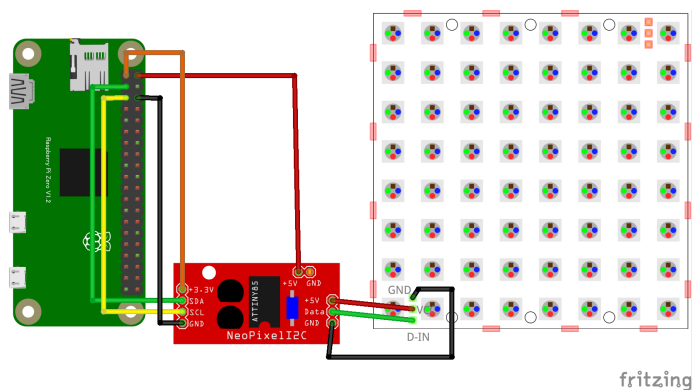


図 2: 配線図（専用ボード使用）

CHIRIMEN ドライバのインストール

```
npm i @chirimen/neopixel-i2c
```

サンプルコード (main.js)

```
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import NPIX from "@chirimen/neopixel-i2c";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

const neoPixels = 7; // LED個数

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const npix = new NPIX(port, 0x41);
  await npix.init(neoPixels);

  await nPixTest1(npix);

  await nPixTest2(npix);

  await nPixTest3(npix, pattern4);

  await npix.setGlobal(0, 0, 0);
}

async function nPixTest1(npix) {
  // 全LEDを同じ色にするケース
  await npix.setGlobal(10, 0, 0);
  await sleep(200);
}
```

```

    await npix.setGlobal(0, 10, 0);
    await sleep(200);
    await npix.setGlobal(0, 0, 10);
    await sleep(200);
    await npix.setGlobal(0, 20, 20);
    await sleep(200);
    await npix.setGlobal(20, 0, 20);
    await sleep(200);
    await npix.setGlobal(20, 20, 0);
    await sleep(200);
    await npix.setGlobal(20, 20, 20);
    await sleep(200);
    await npix.setGlobal(0, 0, 0);
  }

  // パターンはRRGGBB の並びで
  const pattern0 = [0xff0000, 0x00ff00, 0x0000ff, 0xff0000, 0x00ff00, 0x0000ff, 0xff0000];
  const pattern1 = [0x000000, 0x00ff00, 0x0000ff, 0xff0000, 0x00ff00, 0x0000ff, 0xff0000];
  const pattern2 = [0x000000, 0x000000, 0x0000ff, 0xff0000, 0x00ff00, 0x0000ff, 0xff0000];
  const pattern3 = [0x000000, 0x000000, 0x000000, 0xff0000, 0x00ff00, 0x0000ff, 0xff0000];
  const pattern4 = [0x000000, 0x000000, 0x000000, 0x000000, 0x00ff00, 0x0000ff, 0xff0000];
  const pattern5 = [0x000000, 0x000000, 0x000000, 0x000000, 0x000000, 0x0000ff, 0xff0000];
  const pattern6 = [0x000000, 0x000000, 0x000000, 0x000000, 0x000000, 0x000000, 0xff0000];
  const pattern7 = [0x000000, 0x000000, 0x000000, 0x000000, 0x000000, 0x000000, 0x000000];

  async function setPattern(npix, pattern) {
    // パターン設定
    const grbArray = [];
    for (const color of pattern) {

```

```

        const r = color >> 16 & 0xff;
        const g = color >> 8 & 0xff;
        const b = color & 0xff;
        grbArray.push(g);
        grbArray.push(r);
        grbArray.push(b);
    }
    await npix.setPixels(grbArray);
}

async function nPixTest2(npix) {
    // パターンを変化させる
    await setPattern(npix, pattern0);
    await sleep(200);
    await setPattern(npix, pattern1);
    await sleep(200);
    await setPattern(npix, pattern2);
    await sleep(200);
    await setPattern(npix, pattern3);
    await sleep(200);
    await setPattern(npix, pattern4);
    await sleep(200);
    await setPattern(npix, pattern5);
    await sleep(200);
    await setPattern(npix, pattern6);
    await sleep(200);
    await setPattern(npix, pattern7);
}

async function nPixTest3(npix, pattern) {
    // パターンを流す
    for (let i = 0; i < 30; i++) {
        const p = i % pattern.length;
        const spattern = [];
        for (let px = 0; px < pattern.length; px++) {
            const pp = (i + px) % pattern.length;
            spattern.push(pattern[pp]);
        }
    }
}

```

```
    }  
    console.log(spattern);  
    await setPattern(npix, spattern);  
    await sleep(200);  
  }  
}
```

電子ペーパー

WAVESHARE20471複合センサーボード

配線図

説明ページの組み立て方^{*1}に従って Raspberry Pi Zero に接続したら完了です。

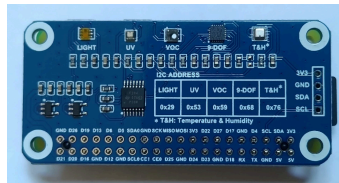


図1: ボードの写真

CHIRIMEN ドライバのインストール

```
npm i @chirimen/bme280
npm i @chirimen/icm20948
npm i @chirimen/ltr390
npm i @chirimen/tsl2591
npm i @chirimen/sgp40
```

1. https://www.waveshare.com/wiki/Environment_Sensor_HAT#Hardware_connection

サンプルコード (main.js)

```
// webI2C
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";

// I2c device drivers
import BME280 from "@chirimen/bme280"; // BME280
import ICM20948 from "@chirimen/icm20948"; // ICM20948
import LTR390 from "@chirimen/ltr390"; // LTR390
import TSL2591 from "@chirimen/tsl2591"; // TSL2591
import SGP40 from "@chirimen/sgp40"; // SGP40

const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

var sgp40, tsl2591, ltr390, icm20948, bme280;

main();

async function main() {
  // initialize I2C
  const i2cAccess = await requestI2CAccess();
  const ic2Port = i2cAccess.ports.get(1);
  // initialize I2C devices

  sgp40 = new SGP40(ic2Port);
  tsl2591 = new TSL2591(ic2Port);
  ltr390 = new LTR390(ic2Port);
  icm20948 = new ICM20948(ic2Port);
  bme280 = new BME280(ic2Port, 0x76); // これだけ0x76忘れずに

  await sgp40.init();
  await tsl2591.init();
  await ltr390.init();
  await icm20948.init();
}
```

```

    await bme280.init();

    // sensing loop
    while (true) {
        const sgp = await sgp40.measureRaw(25, 50);
        const tsl = await tsl2591.Lux();
        const ltr = await ltr390.UVS();
        const icm = await icm20948.getdata();
        const bme = await bme280.readData();

        console.log("=====
=====");
        console.log("Pressure: ", bme.pressure);
        console.log("Temperature: ", bme.temperature);
        console.log("Humidity: ", bme.humidity);
        console.log("Lux: ", tsl);
        console.log("UV: ", ltr);
        console.log("Gas: ", sgp );
        console.log(`Roll = ${icm[0].toFixed(2)} , Pitch = ${icm[
1].toFixed(2)} , Yaw = ${icm[2].toFixed(2)}`);
        console.log(`Acceleration: X = ${icm[3]}, Y = ${icm[4]},
Z = ${icm[5]}`);
        console.log(`Gyroscope:      X = ${icm[6]} , Y = ${icm[7]}
, Z = ${icm[8]}`);
        console.log(`Magnetic:      X = ${icm[9]} , Y = ${icm[10]
} , Z = ${icm[11]}`);

        await sleep(1000);
    }
}

```

モーター制御

ステッピングモータードライバー

A4988ドライバによるステッピングモータ制御

配線図

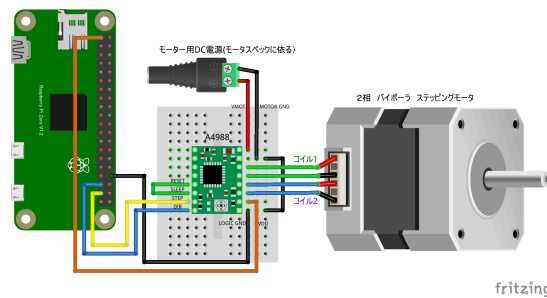


図 1: 配線図

CHIRIMEN ドライバのインストール

- 不要です。

サンプルコード (main.js)

```
import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

let portA, portB;

async function init() {
  const gpioAccess = await requestGPIOAccess();
  portA = gpioAccess.ports.get(26);
  await portA.export("out");
  await portA.write(0);
  portB = gpioAccess.ports.get(19);
  await portB.export("out");
  await portB.write(0);
}

async function main() {
  await init();
  let direction = 1;
  for (; ;) {
    console.log("Move:", direction);
    await stepMove(100, direction);
    await sleep(500);
    direction = direction == 1 ? 0 : 1;
  }
}

async function stepMove(steps, direction) {
  await portB.write(direction);
  for (let i = 0; i < steps; i++) {
    await portA.write(1);
    await sleep(1);
    await portA.write(0);
  }
}
```

```
        await sleep(20);  
    }  
}  
  
main();
```

H ブリッジ

HBridge1

Hブリッジモータードライバは正転[1,0]・逆転[0,1]・ブレーキ[1,1]・フリー[0,0]の4状態をGPIOの2つの信号線を使って指示します

配線図

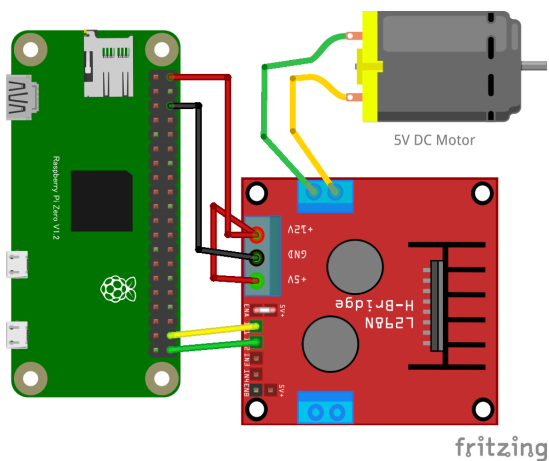


図 1: 配線図

CHIRIMEN ドライバのインストール

- 不要です。

サンプルコード (main.js)

```
// Hブリッジモータードライバは正転[1,0]・逆転[0,1]・ブレーキ[1,1]・フリー[0,0]の4状態を
// GPIOの2つの信号線を使って指示します

import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

const portAddrs = [20, 21]; // HブリッジコントローラをつなぐGPIOポート番号
let ports;

main();

async function main() {
  await init();

  for (; ;) {
    console.log("fwd");
    await fwd();
    await sleep(1000);
    console.log("rev");
    await rev();
    await sleep(1000);
    console.log("brake");
    await brake();
    await sleep(1000);
  }
}

async function init() {
  // ポートを初期化するための非同期関数
  const gpioAccess = await requestGPIOAccess(); // thenの前の関数
```

をawait接頭辞をつけて呼び出します。

```
ports = [];  
  
for (let i = 0; i < 2; i++) {  
    ports[i] = gpioAccess.ports.get(portAddrs[i]);  
    await ports[i].export("out");  
}  
for (let i = 0; i < 2; i++) {  
    ports[i].write(0);  
}  
}  
  
async function free() {  
    ports[0].write(0);  
    ports[1].write(0);  
}  
  
async function brake() {  
    ports[0].write(1);  
    ports[1].write(1);  
    await sleep(300); // 300ms待機してフリー状態にします  
    ports[0].write(0);  
    ports[1].write(0);  
}  
  
async function fwd() {  
    ports[0].write(1);  
    ports[1].write(0);  
}  
  
async function rev() {  
    ports[0].write(0);  
    ports[1].write(1);  
}
```

PCA9685 16 チャンネル PWM ドライバ

- PCA9685 16 チャンネルPWM ドライバーとHBridgeモータードライバーで、DCモーターを正転・逆転・速度コントロールします。
- HBridgeコントローラはL298N以外にもL9110S、MX1508等も同様に使用できます。
 - ただし、これらのドライバでは+5Vと+12V端子の代わりに、一個のVCCや電源(+)端子だけのものが多いので、モーター電源はそこに繋がります。

配線図

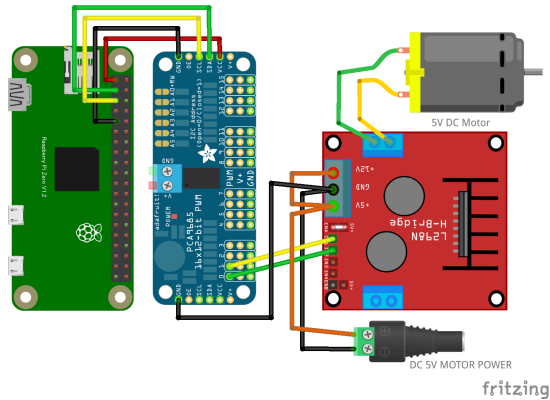


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/pca9685-pwm
```

サンプルコード (main.js)

```
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import PCA9685_PWM from "@chirimen/pca9685-pwm";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

let pca9685pwm;

main();

async function main() {
  await init();

  let direction = -1;
  for (; ;) {
    direction = (direction == 1) ? -1 : 1; // 三項演算子でdirectionを反転
    console.log("direction:", direction);
    console.log("speedUp");
    for (let speed = 0; speed <= 1; speed += 0.1) {
      await setMotor(direction, speed);
      await sleep(200);
    }
    await sleep(500);
    console.log("speedDown");
    for (let speed = 1; speed >= 0; speed -= 0.1) {
      await setMotor(direction, speed);
      await sleep(200);
    }
    console.log("stop");
    await setMotor(0, 0);
    await sleep(1000);
  }
}
```

```
async function init() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  pca9685pwm = new PCA9685_PWM(port, 0x40);
  await pca9685pwm.init(100);
}

async function setMotor(direction, speed) {
  // direction: モーターの回転方向 正転:1,逆転:-1,停止:0
  // speed: モーターのスピード 0:停止...1.0:全速
  if (direction == 1) {
    await pca9685pwm.setPWM(1, 0);
    await pca9685pwm.setPWM(0, speed);
  } else if (direction == -1) {
    await pca9685pwm.setPWM(0, 0);
    await pca9685pwm.setPWM(1, speed);
  } else {
    await pca9685pwm.setPWM(0, 0);
    await pca9685pwm.setPWM(1, 0);
  }
}
```

PWM コントローラー

PCA9685 16 チャンネルサーボモーター PWM ドライバー

配線図

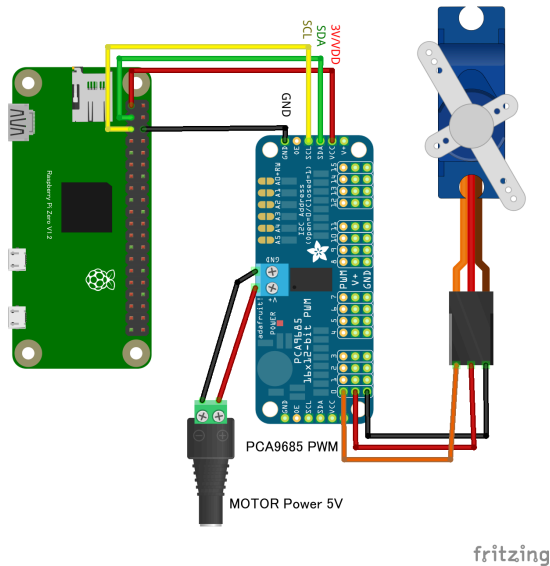


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/pca9685
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import PCA9685 from "@chirimen/pca9685";
```

```
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const pca9685 = new PCA9685(port, 0x40);
  // servo setting for sg90
  // Servo PWM pulse: min=0.0011[sec], max=0.0019[sec] angle=+-60
  [deg]
  await pca9685.init(0.001, 0.002, 30);
  for (;;) {
    await pca9685.setServo(0, -30);
    console.log("-30 deg");
    await sleep(1000);
    await pca9685.setServo(0, 30);
    console.log("30 deg");
    await sleep(1000);
  }
}
```

アナログ・デジタル変換

ADC

ADS1015 12 ビット AD コンバータ

配線図

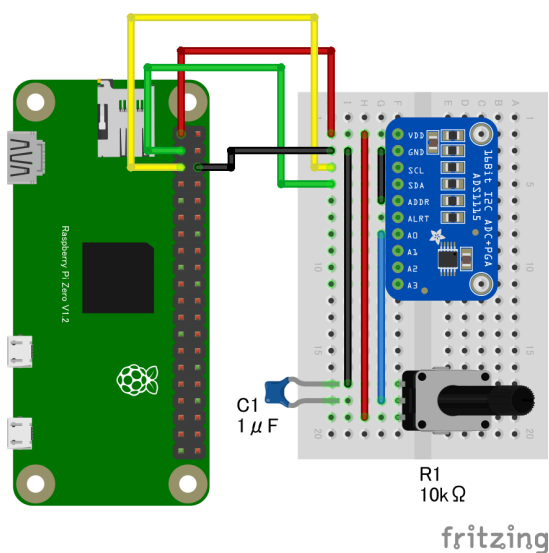


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/ads1015
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index
.js";
import ADS1015 from "@chirimen/ads1015";
const sleep = msec => new Promise(resolve => setTimeout(resolve,
msec));

main();

async function main() {
  var i2cAccess = await requestI2CAccess();
  var port = i2cAccess.ports.get(1);
  var ads1015 = new ADS1015(port, 0x48);
  await ads1015.init();
  for (;;) {
    try {
      var value = await ads1015.read(0);
      console.log("value:", value);
    } catch (error) {
      console.error("error: code:" + error.code + " message:" + e
rror.message);
    }
    await sleep(100);
  }
}
```

ADS1115 16bit ADC 差動入力によるロードセルの使用

配線図

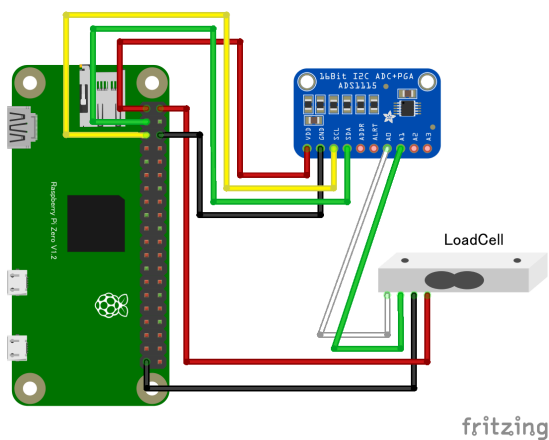


図1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/ads1x15
```

サンプルコード (main.js)

```
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import ADS1X15 from "@chirimen/ads1x15";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const ads1115 = new ADS1X15(port, 0x48);
  // If you uses ADS1115, you have to select "true", otherwise
  select "false".
  await ads1115.init(true, 7); // High Gain
  console.log("init complete");
  var firstTime = true;
  var tare;
  while (true) {
    var difA = await ads1115.read("0,1"); // p0-p1 different
    ial mode
    console.log("dif chA(0-1):" + difA.toString(16) + " : " +
ads1115.getVoltage(difA).toFixed(6) + "V");
    if (firstTime) {
      tare = difA;
      firstTime = false;
    }
    var weight = difA - tare;
    console.log("rawData - Tare:" + weight.toString(16) + " :
" + ads1115.getVoltage(weight).toFixed(6) + "V");
    await sleep(500);
  }
}

```

ADS1115 16bit ADC

配線図

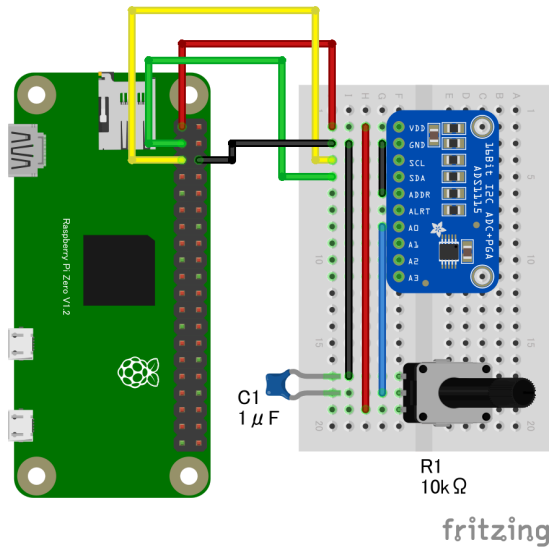


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/ads1x15
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index
.js";
import ADS1X15 from "@chirimen/ads1x15";
const sleep = msec => new Promise(resolve => setTimeout(resolve,
msec));
```

```

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const ads1x15 = new ADS1X15(port, 0x48);
  // If you uses ADS1115, you have to select "true", otherwise select "false".
  await ads1x15.init(true);
  console.log("init complete");
  while (true) {
    let output = "";
    // ADS1115 has 4 channels.
    for (let channel = 0; channel < 4; channel++) {
      const rawData = await ads1x15.read(channel);
      const voltage = ads1x15.getVoltage(rawData);
      output += `CH${channel}:${voltage.toFixed(3)}V `;
    }
    console.log(output);

    await sleep(500);
  }
}

```

PCF8591 8bit AD,DA コンバーター

配線図

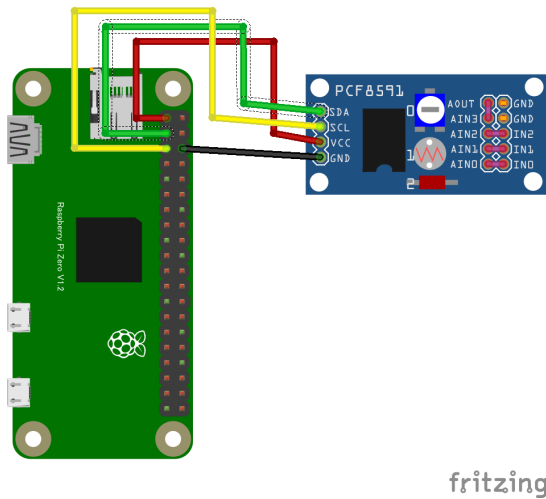


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/pcf8591
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import PCF8591 from "@chirimen/pcf8591";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
```

```
main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const pcf8591 = new PCF8591(port, 0x48);
  await pcf8591.init();

  // Set DAC voltage (Range: 0V - 3.3V)
  await pcf8591.setDAC(3.3);

  while (true) {
    let output = "";

    // PCF8591 has 4 channels
    for (let channel = 0; channel < 4; channel++) {
      const voltage = await pcf8591.readADC(channel);
      output += `CH${channel}:${voltage.toFixed(3)}V `;
    }
    console.log(output);

    await sleep(500);
  }
}
```

電流センサー

INA219 電流センサー

配線図

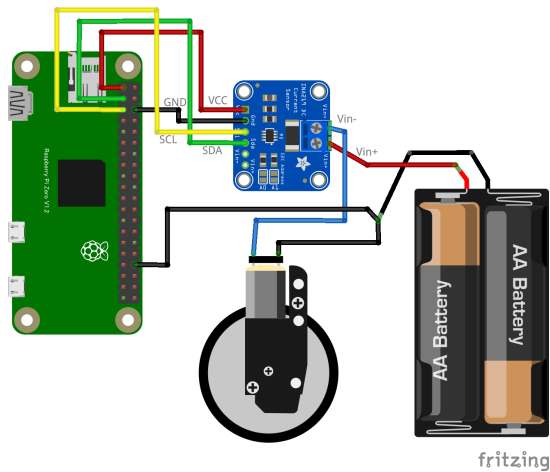


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/ina219
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";
import INA219 from "@chirimen/ina219";
```

```
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const ina219 = new INA219(port, 0x40);
  await ina219.init();
  await ina219.configure();

  while (true) {
    const voltage = await ina219.voltage();
    const supplyVoltage = await ina219.supply_voltage();
    const current = await ina219.current();
    const power = await ina219.power();
    const shuntVoltage = await ina219.shunt_voltage();

    console.log(
      [
        `Voltage: ${voltage.toFixed(3)}V`,
        `Supply voltage: ${supplyVoltage.toFixed(3)}V`,
        `Current: ${current.toFixed(2)}mA`,
        `Power: ${power.toFixed(2)}mW`,
        `Shunt voltage: ${shuntVoltage.toFixed(2)}mV`
      ].join(", ")
    );

    await sleep(500);
  }
}
```

オーディオ

オーディオプレイヤー

DFPlayer Mini

MP3 プレーヤーボードを GPIO OUTPUT で制御

配線図

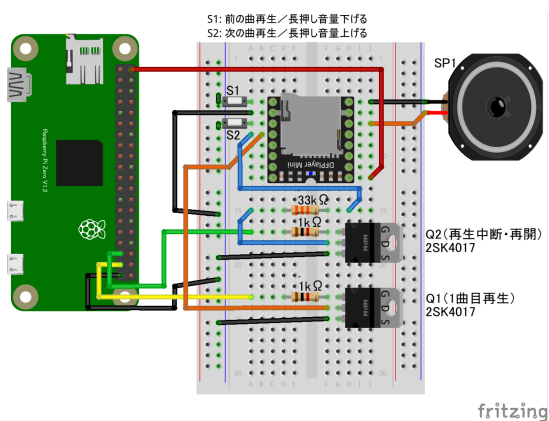


図 1: 配線図

- DFPlayer Mini ボードの ADKEY1 端子を NchMOSFET を介して GPIO で制御します (他にシリアル通信での制御も可能なボードですがこのサンプルは GPIO で制御できる ADKEY 端子を使用)
- 電源投入後、ボリュームが最大になるので S1 スイッチは付けておきましょう

- ADKEY1/2 端子と抵抗を組み合わせることのできる様々なコントロールが可能です。GPIO 端子の使用数を増やすと制御できる種類も増やせます。こちらのページ^{*1}や、メーカーサイトの説明ページ^{*2}を参考に

CHIRIMEN ドライバのインストール

- 不要です。

サンプルコード (main.js)

```
import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

let play1port;
let pauseport;

async function start_play() {
  const gpioAccess = await requestGPIOAccess();
  play1port = gpioAccess.ports.get(26); //
  pauseport = gpioAccess.ports.get(19);

  await play1port.export("out");
  await pauseport.export("out");

  await repeat_play();
}

async function repeat_play() {
  while (true) {
    // 1曲目を再生開始
```

1. <https://chitakekoubou.blogspot.com/p/dfplayeradkeyio.html>

2. https://wiki.dfrobot.com/DFPlayer_Mini_SKU_DFR0299

```
console.log("start play1")
  await play1port.write(1);
  await sleep(300); // 0.3秒保持して元に戻す
  await play1port.write(0);

  // 10秒保持
  await sleep(10 * 1000);

  // 一時停止
  console.log("stop")
  await pauseport.write(1);
  await sleep(300);
  await pauseport.write(0);

  // 3秒間一時停止
  await sleep(3 * 1000);
}
}

start_play();
```

音声録音

ISD1820ボイスレコーダ (GPIO OUTPUT)

配線図

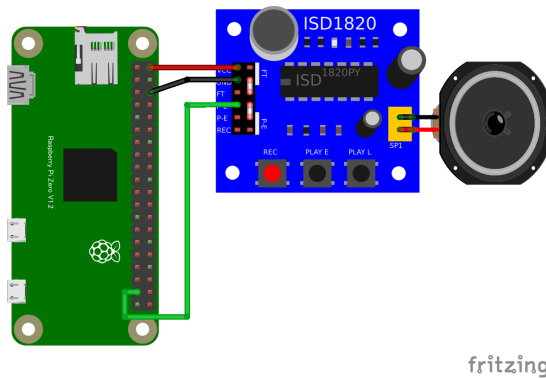


図 1: 配線図

- GPIO PORT26 を ISD1820 の P-L 端子に繋がします(P-L 端子が High になっている間だけ再生します(PLAY L ボタンと同じ動作))
- P-E 端子に繋いだ場合、録音した内容を全部再生するようになります。(PLAY E ボタンと同じ動作)
- 録音は REC ボタンを押している間録音できます(最大 10 秒)
- REC 端子を GPIO に接続すると、録音をコンピュータで制御することもできます。(REC 端子が High になっている間だけ録音)

CHIRIMEN ドライバのインストール

- 不要です。

サンプルコード (main.js)

```
import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

async function blink() {
  const gpioAccess = await requestGPIOAccess();
  const port = gpioAccess.ports.get(26);

  await port.export("out");

  for (;;) {
    await port.write(1);
    await sleep(3000); // 3秒間再生
    await port.write(0);
    await sleep(1000); // 1秒間停止
  }
}

blink();
```

その他

GPS

Serial GPS

PiZero のシリアル端子に、GY-GPS6MV2 等の GPS レシーバ基板を繋いで使用する

- OS の設定
 - `sudo raspi-config`
 - Interface Options -> Serial Port -> Login over serial: いいえ , serial port enabled: はい -> Finish (reboot)
 - Note: この設定は USB シリアルのコンソールログインには影響しない
- 結線 (GPS の RX 端子の結線は基本動作では不要) : 下図参照
 - 動作検証したモジュール (GY-NEO6MV2, 基板の印刷は GY-GPS6MV2)
 - <https://electronicwork.shop/items/625c1ca99fe3d707d725cbe1>^{*1}
 - 同等品と考えられるもの
 - <https://www.amazon.co.jp/dp/B07LF6KGR8>^{*2}
 - <https://www.aitendo.com/product/10255>^{*3}
- 動作確認
 - `cat /dev/ttyS0`
 - Note: GPS 衛星電波受信されていなくてもメッセージが出力される。
`/dev/serial0` も使える。測位成功するとメッセージが派手になり、LED が点滅する (LED は測位成功していないときは消灯)
- シリアルポート及び GPS のライブラリを導入(myApp ディレクトリで)
 - `cd myApp`
 - `npm install serialport gps`
- アプリの実行
 - main.js を myApp 下に保存

1. <https://electronicwork.shop/items/625c1ca99fe3d707d725cbe1>

2. <https://www.amazon.co.jp/dp/B07LF6KGR8>

3. <https://www.aitendo.com/product/10255>

- `node main.js`

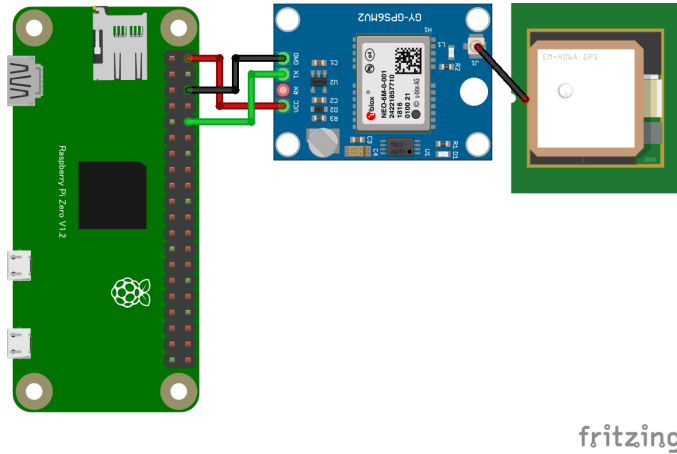


図 1: GY-NEO6MV2.png

CHIRIMEN ドライバのインストール

- 不要です。

サンプルコード (main.js)

```
// シリアルからGPSデータ受信
import { SerialPort } from 'serialport';
import { ReadlineParser } from '@serialport/parser-readline';
import GPSPackage from 'gps';
const GPS = GPSPackage.GPS;

const port = new SerialPort({ path: '/dev/ttyS0', baudRate: 9600
})
const parser = port.pipe(new ReadlineParser())
const gps = new GPS();

parser.on('data', function (txtData) {
  gps.update(txtData);
});
```

```
});

gps.on('data', function (data) {
  if (data.type == "GGA") { // "RMC"タイプデータを読むと速度(ノット)
    が得られる
    console.log(data);
  }
  /**
  console.log("data:", data);
  console.log("stat:", gps.state);
  console.log("=====
=====");
  **/
});
```

サーマルカメラ

AMG8833 サーモグラフィー

配線図

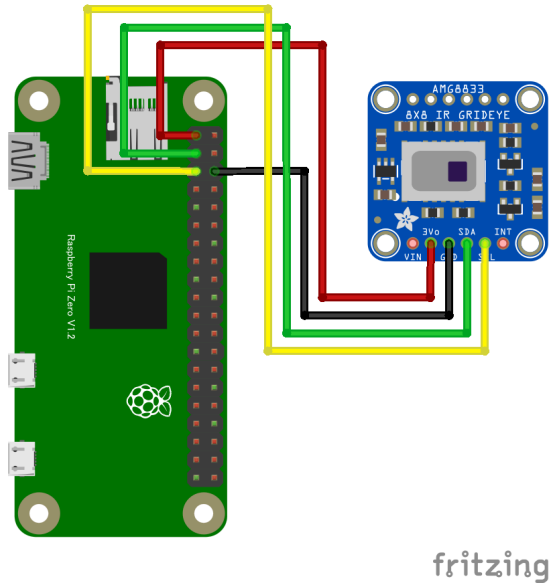


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/amg8833
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index.js";  
import AMG8833 from "@chirimen/amg8833";
```

```
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const amg8833 = new AMG8833(port, 0x69);
  await amg8833.init();

  while (true) {
    const data = await amg8833.readData();
    for (const row of data) {
      // degree Celsius
      console.log(row.map(value => value.toFixed(2)).join(" "));
    }

    console.log();
    await sleep(500);
  }
}
```

赤外線温度センサー

MLX90614 赤外線温度センサー

配線図

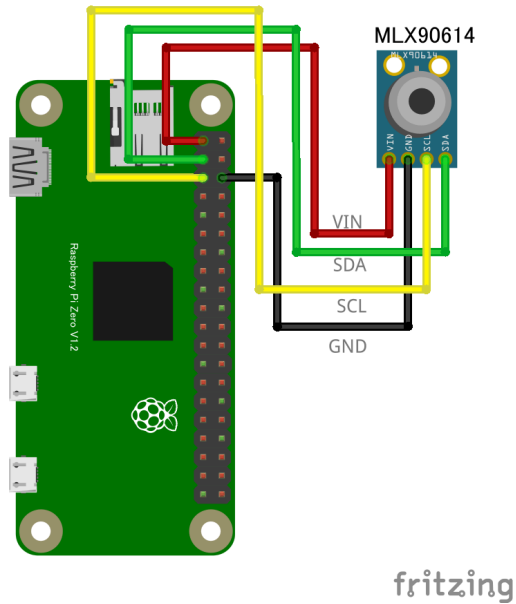


図 1: 配線図

CHIRIMEN ドライバのインストール

```
npm i @chirimen/mlx90614
```

サンプルコード (main.js)

```
import {requestI2CAccess} from "../node_modules/node-web-i2c/index
.js";
import MLX90614 from "@chirimen/mlx90614";
```

```

const sleep = msec => new Promise(resolve => setTimeout(resolve,
msec));

main();

async function main() {
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  const mlx90614 = new MLX90614(port, 0x5a);
  await mlx90614.init();

  while (true) {
    // Temperature of object that the sensor looking at. (Measure
d by IR sensor)
    const objectTemperature = await mlx90614.get_obj_temp();
    // Temperature measured by the chip. (The package temperature
)
    const ambientTemperature = await mlx90614.get_amb_temp();
    console.log(
      [
        `Object temperature: ${objectTemperature.toFixed(2)} degr
ee`,
        `Ambient temperature: ${ambientTemperature.toFixed(2)} de
gree`
      ].join(", ")
    );

    await sleep(500);
  }
}

```

リモート制御

カメラ

GPIOスイッチによるカメラ撮影 (remote_Camera)

配線図

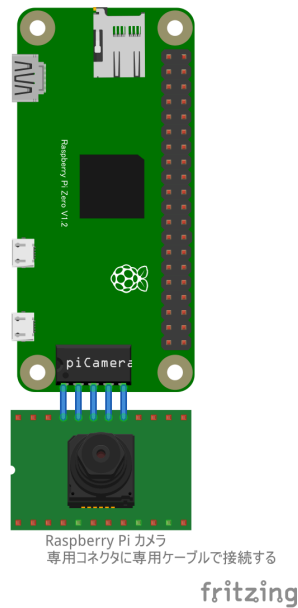


図 1: 配線図

タクトスイッチはGPIO PORT5 に繋がります。

カメラは専用コネクタに専用ケーブルを使って接続し、更にセットアップが必要です。[こちらを参照してください](#)

CHIRIMEN ドライバのインストール

- 不要です。

サンプルコード (main.js)

```
// Raspberry Pi Camera リモートカメラ

// ライブラリ pi-camera-connect をまずインストールする必要があります。readmeを参照してください。

import { StillCamera } from "pi-camera-connect";
import * as fs from "fs";

import { requestGPIOAccess } from "./node_modules/node-web-gpio/dist/index.js";
const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

import nodeWebSocketLib from "websocket"; // https://www.npmjs.com/package/websocket
import { RelayServer } from "./RelayServer.js";

const stillCamera = new StillCamera({
  width: 128,
  height: 128,
});

var channel;

async function connect() {
  // websocketリレーの初期化
  var relay = RelayServer(
    "chirimentest",
    "chirimenSocket",
    nodeWebSocketLib,
    "https://chirimen.org"
  );
  channel = await relay.subscribe("chirimenCAM");
  console.log("web socketリレーサービスに接続しました");
}
```

```
channel.onmessage = transmitImageData;
}

async function transmitImageData(message) {
  console.log(message.data);
  if (message.data == "GET IMAGE DATA") {
    const imageURI = await captureImage();
    var sensorData = {
      imageURI: imageURI,
      time: new Date().getTime(),
    };
    channel.send(sensorData);
    console.log("Send ImageData: length:", JSON.stringify(sensorData).length);
  }
}

// カメラの画像をDataURLとして取得する
// このAPIについてはreadme.mdを参照してください。
async function captureImage() {
  var mime = "image/jpeg";
  var encoding = "base64";
  const image = await stillCamera.takeImage();
  const b64str = image.toString(encoding);
  const dataURL = "data:" + mime + ";" + encoding + "," + b64str;
  return dataURL;
}

connect();
```

GPIO

リモートLチカ

配線図

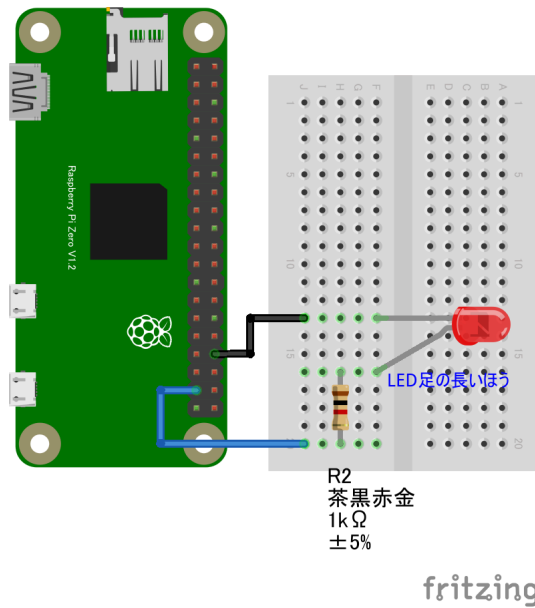


図 1: 配線図

GPIO PORT26 に繋がります

遠隔コントロール(PC/スマホブラウザ)側

pc/index.html^{*1} を起動します。

1. https://codesandbox.io/s/github/chirimen-oh/chirimen.org/tree/master/pizero/src/esm-examples/remote_gpio_led/pc?module=pc.js

CHIRIMEN ドライバのインストール

- 不要です。

サンプルコード (main.js)

```
// Remote Example1 - reciever
// for CHIRIMEN with nodejs

import {requestGPIOAccess} from "./node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
import nodeWebSocketLib from "websocket"; // https://www.npmjs.com/package/websocket
import {RelayServer} from "./RelayServer.js";

var channel;
var gpioPort0;

async function connect(){
  // GPIOポート0の初期化
  var gpioAccess = await requestGPIOAccess();
  var mbGpioPorts = gpioAccess.ports;
  gpioPort0 = mbGpioPorts.get(26);
  await gpioPort0.export("out"); //port0 out

  // websocketリレーの初期化
  var relay = RelayServer("chirimentest", "chirimenSocket" , nodeWebSocketLib, "https://chirimen.org");
  channel = await relay.subscribe("chirimenLED");
  console.log("web socketリレーサービスに接続しました");
  channel.onmessage = controlledLED;
}

function controlledLED(messge){
```

```
console.log(messge.data);  
if ( messge.data == "LED ON"){  
    gpioPort0.write(1);  
    console.log("ON");  
    channel.send("LEDをオンにしました");  
} else if ( messge.data == "LED OFF"){  
    gpioPort0.write(0);  
    console.log("OFF");  
    channel.send("LEDをオフにしました");  
}  
}  
  
connect();
```

リモートオンオフセンサ

配線図

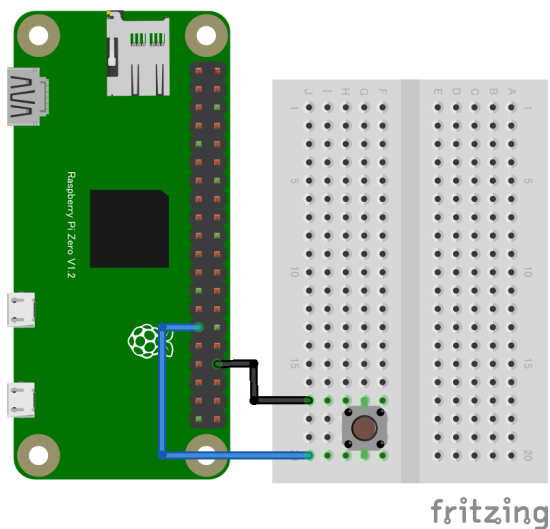


図 1: 配線図

GPIO PORT26 に繋がます

遠隔モニタ(PC/スマホブラウザ)側

pc/index.html^{*1} を起動します。

CHIRIMEN ドライバのインストール

- 不要です。

1. https://codesandbox.io/s/github/chirimen-oh/chirimen.org/tree/master/pizero/src/esm-examples/remote_gpio_sw/pc?module=pc.js

サンプルコード (main.js)

```
// Remote Example9 - reciever
// for CHIRIMEN with nodejs

import {requestGPIOAccess} from "../node_modules/node-web-gpio/dist/index.js";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
import nodeWebSocketLib from "websocket"; // https://www.npmjs.com/package/websocket
import {RelayServer} from "../RelayServer.js";

var channel;
var gpioPort0;

async function connect(){
  // GPIOポート0の初期化
  var gpioAccess = await requestGPIOAccess();
  var mbGpioPorts = gpioAccess.ports;
  gpioPort0 = mbGpioPorts.get(5);
  await gpioPort0.export("in"); //port0 out

  // websocketリレーの初期化
  var relay = RelayServer("chirimentest", "chirimenSocket" , nodeWebSocketLib, "https://chirimen.org");
  channel = await relay.subscribe("chirimenSW");
  console.log("web socketリレーサービスに接続しました");
  gpioPort0.onchange=testChange; // ISSUE gpioのonchangeの仕様が異なる
}

function testChange(val){
  var msgTxt = (val.value === 1 ) ? "High" : "Low"; // 条件 (三項) 演算子
  console.log(msgTxt);
}
```

```
        channel.send(msgTxt);  
    }  
  
    connect();
```

H ブリッジ

リモートモータ

配線図

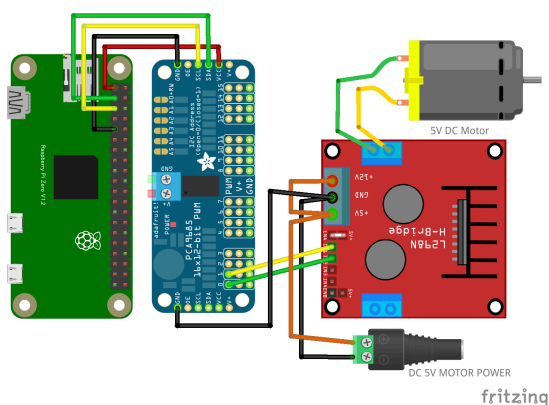


図 1: 配線図

遠隔コントローラ(PC/スマホブラウザ)側

pc/index.html^{*1}を起動します。

1. https://codesandbox.io/s/github/chirimen-oh/chirimen.org/tree/master/pizero/src/esm-examples/remote_hbridge_pca9685/pc?module=pc.js

CHIRIMEN ドライバのインストール

```
npm i @chirimen/pca9685-pwm
```

サンプルコード (main.js)

```
// Remote Example PCA9685PWM + HBridge - reciever
// for CHIRIMEN with nodejs

import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import PCA9685_PWM from "@chirimen/pca9685-pwm";

import nodeWebSocketLib from "websocket"; // https://www.npmjs.com/package/websocket
import { RelayServer } from "../RelayServer.js";

const sleep = (msec) => new Promise((resolve) => setTimeout(resolve, msec));

let pca9685pwm;
let channel;

connect();

async function connect() {
  // I2C PCA9685モジュール (PWM) の初期化
  const i2cAccess = await requestI2CAccess();
  const port = i2cAccess.ports.get(1);
  pca9685pwm = new PCA9685_PWM(port, 0x40);
  await pca9685pwm.init(100);

  // websocketリレーの初期化
  //
  var relay = RelayServer("chirimentest", "chirimenSocket", nodeW
```

```

ebSocketLib,"https://chirimen.org");
    channel = await relay.subscribe("chirimenPWMMHB");
    console.log("web socketリレーサービスに接続しました");
    channel.onmessage = controlMotor;
}

async function controlMotor(message) {
    console.log(message.data);
    if ( message.data.motor==undefined || message.data.speed==und
efined ){
        return;
    }
    var motorNum = message.data.motor;
    var motorSpeed = message.data.speed;
    var direction = 0;
    if (motorSpeed > 0) {
        direction = 1;
    } else if (motorSpeed == 0) {
        direction = 0;
    } else {
        direction = -1;
    }
    var speed = Math.min(1, Math.abs(motorSpeed));
    await setMotor(direction, speed);
    console.log("MOTOR" + motorNum + "を" + direction * speed + "
に設定しました");
    channel.send({motor: motorNum, setSpeed: (speed*direction)});
}

async function setMotor(direction, speed) {
    // direction: モーターの回転方向 正転:1,逆転:-1,停止:0
    // speed: モーターのスピード 0:停止...1.0:全速
    if (direction == 1) {
        await pca9685pwm.setPWM(1, 0);
        await pca9685pwm.setPWM(0, speed);
    } else if (direction == -1) {
        await pca9685pwm.setPWM(0, 0);
    }
}

```

```
        await pca9685pwm.setPWM(1, speed);  
    } else {  
        await pca9685pwm.setPWM(0, 0);  
        await pca9685pwm.setPWM(1, 0);  
    }  
}
```

PWM

リモートサーボモータ

配線図

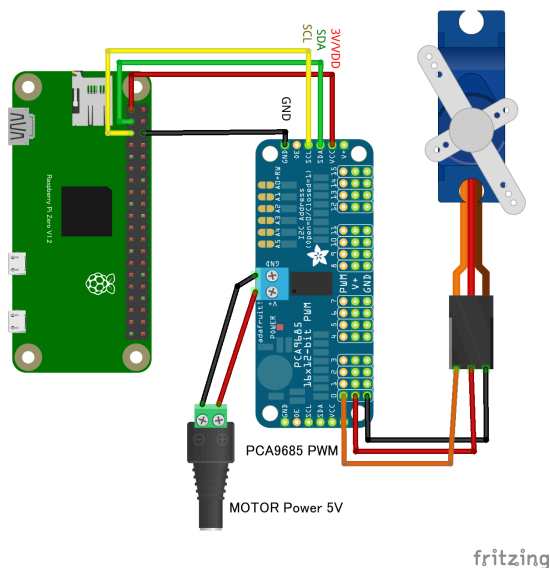


図 1: 配線図

遠隔コントローラ(PC/スマホブラウザ)側

pc/index.html^{*1}を起動します。

1. https://codesandbox.io/s/github/chirimen-oh/chirimen.org/tree/master/pizero/src/esm-examples/remote_pca9685/pc?module=pc.js

CHIRIMEN ドライバのインストール

```
npm i @chirimen/pca9685
```

サンプルコード (main.js)

```
// Remote Example8 - reciever
// for CHIRIMEN with nodejs

import {requestI2CAccess} from "./node_modules/node-web-i2c/index
.js";
import PCA9685 from "@chirimen/pca9685";
const sleep = msec => new Promise(resolve => setTimeout(resolve,
msec));
import nodeWebSocketLib from "websocket"; // https://www.npmjs.co
m/package/websocket
import {RelayServer} from "./RelayServer.js";

var channel;
var pca9685;

async function connect(){
  // chirimen with micro:bit、サーボコントローラの初期化
  var i2cAccess = await requestI2CAccess();
  var i2cPort = i2cAccess.ports.get(1);
  pca9685 = new PCA9685(i2cPort, 0x40);
  await pca9685.init(0.001, 0.002, 30);

  // websocketリレーの初期化
  var relay = RelayServer("chirimentest", "chirimenSocket" , no
deWebSocketLib, "https://chirimen.org");
  channel = await relay.subscribe("chirimenMbitRemoteServo");
  console.log("web socketリレーサービスに接続しました");
  channel.onmessage=moveServo;
```

```
}

async function moveServo(message) {
//  console.log("message:",message);
  if ( !message.data.slope){
    return;
  }
  var angle = message.data.slope;
  if ( Math.abs(angle)>28){
    angle = Math.sign(angle)*28;
  }
  console.log("servo:",angle);
  await pca9685.setServo(0, angle);
  console.log("サーボ角度:"+angle);
  channel.send({setAngle:angle});
}

connect();``
```

センサー

リモート温湿度センサ

配線図

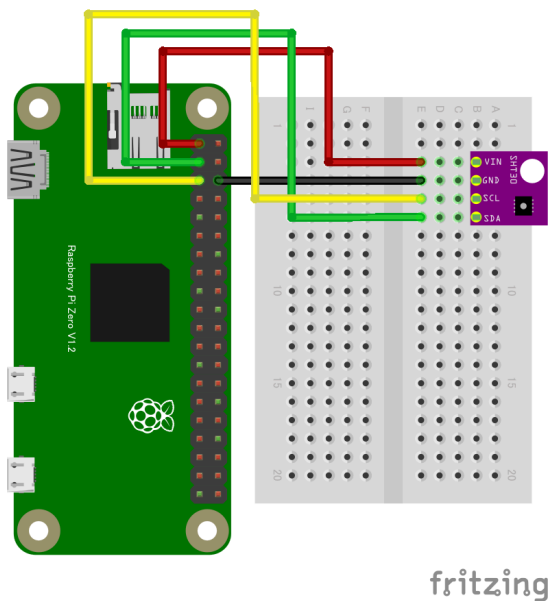


図 1: 配線図

遠隔モニタ(PC/スマホブラウザ)側

pc/index.html^{*1}を起動します。

1. https://codesandbox.io/s/github/chirimen-oh/chirimen.org/tree/master/pizero/src/esm-examples/remote_sht30/pc?module=pc.js

CHIRIMEN ドライバのインストール

```
npm i @chirimen/sht30
```

サンプルコード (main.js)

```
// Remote Example5 - reciever
// for CHIRIMEN with nodejs
import {requestI2CAccess} from "../node_modules/node-web-i2c/index
.js";
import SHT30 from "@chirimen/sht30";
const sleep = msec => new Promise(resolve => setTimeout(resolve,
msec));
import nodeWebSocketLib from "websocket"; // https://www.npmjs.co
m/package/websocket
import {RelayServer} from "../RelayServer.js";

var channel;
var sht;

async function connect(){
  // I2Cポートと、I2CデバイスSHT30の初期化
  var i2cAccess = await requestI2CAccess();
  var i2cPort = i2cAccess.ports.get(1);
  sht = new SHT30(i2cPort);
  await sht.init();

  // websocketリレーの初期化
  var relay = RelayServer("chirimentest", "chirimenSocket" , no
deWebSocketLib, "https://chirimen.org");
  channel = await relay.subscribe("chirimenSHT");
  console.log("web socketリレーサービスに接続しました");
  channel.onmessage = transmitSensorData;
}
```

```
async function transmitSensorData(messge){
  console.log(messge.data);
  if ( messge.data == "GET SENSOR DATA"){
    var sensorData = await readData();
    channel.send(sensorData);
    console.log(JSON.stringify(sensorData));
  }
}

async function readData(){
  var shtData = await sht.readData();
  console.log('shtData:', shtData);
  console.log("temperature:" + shtData.temperature + "degree <
br>humidity:" + shtData.humidity + "%");
  return(shtData);
}

connect();``
```

リモート温湿度センサ2

スイッチを押すと温湿度センサーの値を送信します。スイッチの代わりに人感センサー等を繋ぐと有意義かもしれません。

配線図

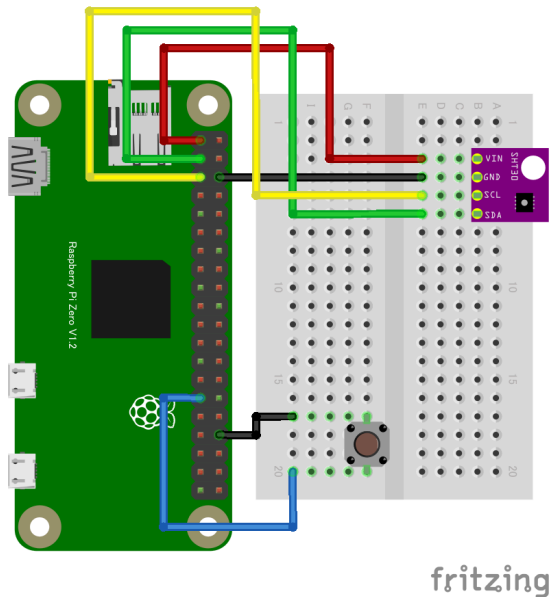


図 1: 配線図

遠隔モニタ(PC/スマホブラウザ)側

pc/index.html^{*1}を起動します。

1. https://codesandbox.io/s/github/chirimen-oh/chirimen.org/tree/master/pizero/src/esm-examples/remote_sht30_gpioin/pc?module=pc.js

CHIRIMEN ドライバのインストール

```
npm i @chirimen/sht30
```

サンプルコード (main.js)

```
// Remote Example5 - reciever
// for CHIRIMEN with nodejs
import { requestGPIOAccess } from "../node_modules/node-web-gpio/dist/index.js";
import { requestI2CAccess } from "../node_modules/node-web-i2c/index.js";
import SHT30 from "@chirimen/sht30";
const sleep = msec => new Promise(resolve => setTimeout(resolve, msec));
import nodeWebSocketLib from "websocket"; // https://www.npmjs.com/package/websocket
import { RelayServer } from "../RelayServer.js";

var channel;
var sht;

async function connect() {
  // GPIOポートの初期化
  const gpioAccess = await requestGPIOAccess();
  const port = gpioAccess.ports.get(5);
  await port.export("in");

  // I2Cポートと、I2CデバイスSHT30の初期化
  var i2cAccess = await requestI2CAccess();
  var i2cPort = i2cAccess.ports.get(1);
  sht = new SHT30(i2cPort);
  await sht.init();

  // websocketリレーの初期化
```

```

    var relay = RelayServer("chirimentest", "chirimenSocket", nodeWebSocketLib, "https://chirimen.org");
    channel = await relay.subscribe("chirimenSHT");
    console.log("web socketリレーサービスに接続しました");
    port.onchange = transmitSensorData;
}

async function transmitSensorData(ev) {
    console.log(ev.value);
    if (ev.value == 0) {
        var sensorData = await readData();
        channel.send(sensorData);
        console.log(JSON.stringify(sensorData));
    }
}

async function readData() {
    var shtData = await sht.readData();
    return (shtData);
}

connect();``

```

リモート距離センサ

配線図

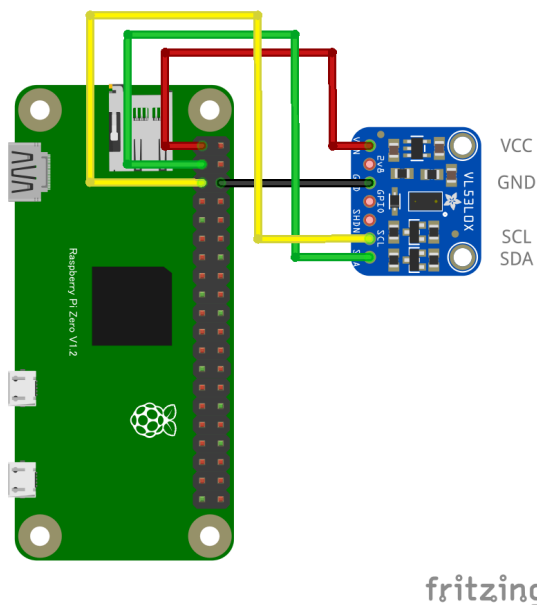


図 1: 配線図

遠隔モニタ(PC/スマホブラウザ)側

pc/index.html^{*1}を起動します。

CHIRIMEN ドライバのインストール

```
npm i @chirimen/vl53l0x
```

1. https://codesandbox.io/s/github/chirimen-oh/chirimen.org/tree/master/pizero/src/esm-examples/remote_vl53l0x/pc?module=pc.js

サンプルコード (main.js)

```

// Remote VL53L0X
// for CHIRIMEN with nodejs
import {requestI2CAccess} from "../node_modules/node-web-i2c/index
.js";
import VL53L0X from "@chirimen/vl53l0x";
const sleep = msec => new Promise(resolve => setTimeout(resolve,
msec));
import nodeWebSocketLib from "websocket"; // https://www.npmjs.co
m/package/websocket
import {RelayServer} from "../RelayServer.js";

var channel;
var vl;

async function connect(){
  // I2Cポートと、I2CデバイスVL53L0Xの初期化
  var i2cAccess = await requestI2CAccess();
  var i2cPort = i2cAccess.ports.get(1);
  vl = new VL53L0X(i2cPort, 0x29);
  await vl.init();

  // websocketリレーの初期化
  var relay = RelayServer("chirimentest", "chirimenSocket" , no
deWebSocketLib, "https://chirimen.org");
  channel = await relay.subscribe("chirimenVL");
  console.log("web socketリレーサービスに接続しました");
  transmitSensorData();
}

async function transmitSensorData(){
  for (;;){
    var sensorData = await readData();
    channel.send(sensorData);
    await sleep(1000);
  }
}

```

```
    }  
  }  
  
  async function readData(){  
    var vlData = await vl.getRange();  
    console.log("distance:" + vlData + "mm");  
    return(vlData);  
  }  
  
  connect();
```



Special Thanks

表紙・イラスト

- @天灯せいた^{*1}

Works

- @gurezo^{*2}
- @kazygumi^{*3}
- @satakagi^{*4}
- @sizuhiko^{*5}

1. <https://www.pixiv.net/users/31028725>

2. <https://github.com/gurezo>

3. <https://github.com/kazygumi>

4. <https://github.com/satakagi>

5. <https://twitter.com/sizuhiko>